

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных Технологий
Кафедра Программной инженерии
Специальность 6-05-0612-01 Программная инженерия

**ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОМУ ПРОЕКТУ НА ТЕМУ:**

«Разработка компилятора SAA-2025»

Выполнил студент Сикорский Артём Александрович
(Ф.И.О.)

Руководитель проекта преп.-стажер Авдеева Вера Дмитриевна
(учен. степень, звание, должность, подпись, Ф.И.О.)

Заведующий кафедрой к.т.н., доц. Смелов В.В.
(учен. степень, звание, должность, подпись, Ф.И.О.)

Консультанты преп.-стажер Авдеева Вера Дмитриевна
(учен. степень, звание, должность, подпись, Ф.И.О.)

Курсовой проект защищен с оценкой _____

Минск 2025

Содержание

Введение.....	4
1 Спецификация языка программирования.....	5
1.1 Характеристика языка программирования.....	5
1.2 Определение алфавит языка программирования.....	5
1.3 Применяемые сепараторы.....	5
1.4 Применяемые кодировки.....	6
1.5 Типы данных.....	7
1.6 Преобразование типов данных.....	7
1.7 Идентификаторы.....	7
1.8 Литералы.....	8
1.9 Объявление данных.....	9
1.10 Инициализация данных.....	9
1.11 Инструкции языка.....	9
1.12 Операции языка.....	10
1.13 Выражения и их вычисления.....	11
1.14 Конструкции языка.....	11
1.15 Область видимости идентификаторов.....	12
1.16 Семантические проверки.....	12
1.17 Распределение оперативной памяти на этапе выполнения.....	12
1.18 Стандартная библиотека и ее состав.....	12
1.19 Ввод и вывод данных.....	13
1.20 Точка входа.....	13
1.21 Препроцессор.....	14
1.22 Соглашения о вызовах.....	14
1.23 Объектный код.....	14
1.24 Классификация сообщений транслятора.....	14
1.25 Контрольный пример.....	14
2 Структура транслятора.....	15
2.1 Компоненты транслятора, их назначение и принципы взаимодействия.....	15
2.2 Перечень параметров транслятора.....	16
2.3 Перечень протоколов, формируемых транслятором.....	16
3 Разработка лексического анализатора.....	17
3.1 Структура лексического анализатора.....	17
3.2 Входные и выходные данные лексического анализатора.....	17
3.3 Параметры лексического анализатора.....	18
3.4 Алгоритм лексического анализа.....	18
3.5 Контроль входных символов.....	20
3.6 Удаление избыточных символов.....	20
3.7 Перечень ключевых слов.....	21
3.8 Основные структуры данных.....	22

3.9 Структура и перечень сообщений лексического анализатора	22
3.10 Принцип обработки ошибок	23
3.11 Контрольный пример	23
4 Разработка синтаксического анализатора	24
4.1 Структура синтаксического анализатора	24
4.2 Контекстно свободная грамматика, описывающая синтаксис языка	24
4.3 Построение конечного магазинного автомата	26
4.4 Основные структуры данных.....	27
4.5 Описание алгоритма синтаксического разбора	27
4.6 Параметры синтаксического анализатора	28
4.7 Структура и перечень сообщений синтаксического анализатора.....	28
4.8 Принцип обработки ошибок	28
4.9 Контрольный пример	29
5 Разработка семантического анализатора	30
5.1 Структура семантического анализатора	30
5.2 Функции семантического анализатора	30
5.3 Структура и перечень сообщений семантического анализатора	31
5.4 Принцип обработки ошибок	32
5.5 Контрольный пример	32
6.1 Выражения, допускаемые языком.....	33
6.2 Польская запись и принцип ее построения	33
6.3 Программная реализация обработки выражений	34
6.4 Контрольный пример	34
7 Генерация кода	35
7.1 Структура генератора кода	35
7.2 Представление типов данных в оперативной памяти	35
7.3 Статическая библиотека	36
7.4 Особенности алгоритма генерации кода	37
7.5 Параметры, управляющие генерацией кода.....	38
7.6 Контрольный пример	38
8 Тестирование транслятора	39
8.1 Тестирование фазы проверки на допустимость символов	39
8.2 Результаты тестирования	39
Заключение	41
Список использованных источников	42
ПРИЛОЖЕНИЕ А	43
ПРИЛОЖЕНИЕ Б.....	45
ПРИЛОЖЕНИЕ В	46
ПРИЛОЖЕНИЕ Г	79
ПРИЛОЖЕНИЕ Е.....	86
ПРИЛОЖЕНИЕ Ж	91

Введение

Задачей данного курсового проекта является разработка транслятора для своего языка программирования: SAA-2025.

Написание транслятора будет осуществляться на языке C++, при этом код на языке SAA-2025 будет транслироваться в язык ассемблера.

Транслятор SAA-2025 состоит из следующих частей:

- лексический анализатор;
- синтаксический анализатор;
- семантический анализатор;
- генератор исходного кода на языке ассемблера.

Исходя из цели курсового проекта, были определены следующие задачи:

- разработка спецификации языка программирования;
- разбратка структуры транслятора;
- разработка лексического и семантического анализаторов;
- разработка синтаксического анализатора;
- преобразование выражений;
- генерация кода на язык ассемблера;
- тестирование транслятора.

Решения каждой из поставленных задач будут приведены в соответствующих главах курсового проекта.

1 Спецификация языка программирования

1.1 Характеристика языка программирования

Язык SAA-2025 – это высокоуровневый язык программирования. Он является процедурным, компилируемым, не поддерживает объектно-ориентированный подход. Язык является строго типизируемым, поэтому преобразование типов не поддерживается.

1.2 Определение алфавит языка программирования

Совокупность символов, используемых в языке, называется алфавитом языка. На этапе выполнения могут использоваться символы латинского алфавита, цифры десятичной системы счисления от 0 до 9, спецсимволы, а также непечатные символы пробела, табуляции и перевода строки. Использование русских символов разрешено только в строковых литералах.

1.3 Применяемые сепараторы

Символы-сепараторы служат в качестве разделителей операций языка. Символы, используемые в качестве сепараторов в языке программирования SAA-2025, и их область применения приведены в таблице 1.1.

Таблица 1.1 – Сепараторы

Сепаратор	Название	Область применения
1	2	3
‘ ‘	Пробел	Допускается везде, кроме идентификаторов и ключевых слов
;	Точка с запятой	Разделение конструкций
{...}	Фигурные скобки	Заключение программного блока
[...]	Квадратные кавычки	Блок кода
(...)	Круглые скобки	Приоритет операций, параметры функции
“...”	Двойные кавычки	Строковый литерал
‘...’	Одинарные кавычки	Допускается везде, кроме идентификаторов и ключевых слов
=	Знак «равно»	Присваивание значения
,	Запятая	Разделение параметров
/ \ + - * :	Знаки «косая черта», «обратная косая черта», «плюс», «минус», «звездочка», «двоеточие»	Выражения
% & 	Знаки «процент», «амперсанд», «вертикальная линия»	Выражения в операторе цикла и условном операторе

Продолжение таблицы 1.2

1	2	3
~	Знаки «тильде»	Выражения
! ^ < >	«восклицательный знак», «циркумфлекс», «больше» и «меньше»	Выражения в операторе цикла и условном операторе

Исходя из данных таблицы, в языке присутствует достаточно различных сепараторов с разной областью применения, что делает язык гибким в использовании.

1.4 Применяемые кодировки

Для написания исходного кода на языке программирования SAA-2025 используется кодировка Windows-1251.

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00	<u>NUL</u> 0000	<u>STX</u> 0001	<u>SOT</u> 0002	<u>ETX</u> 0003	<u>EOT</u> 0004	<u>ENQ</u> 0005	<u>ACK</u> 0006	<u>BEL</u> 0007	<u>BS</u> 0008	<u>HT</u> 0009	<u>LF</u> 000A	<u>VT</u> 000B	<u>FF</u> 000C	<u>CR</u> 000D	<u>SO</u> 000E	<u>SI</u> 000F
10	<u>DLE</u> 0010	<u>DC1</u> 0011	<u>DC2</u> 0012	<u>DC3</u> 0013	<u>DC4</u> 0014	<u>NAK</u> 0015	<u>SYN</u> 0016	<u>ETB</u> 0017	<u>CAN</u> 0018	<u>EM</u> 0019	<u>SUB</u> 001A	<u>ESC</u> 001B	<u>FS</u> 001C	<u>GS</u> 001D	<u>RS</u> 001E	<u>US</u> 001F
20	<u>SP</u> 0020	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
30	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
40	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
50	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
60	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
70	p	q	r	s	t	u	v	w	x	y	z	{		}	~	<u>DEL</u> 007F
80	Ъ	Ѓ	Ѕ	Ї	Њ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ
90	Ѓ	Ѕ	Ї	Њ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ
A0	<u>MBSP</u> 00A0	Ѓ	Ѕ	Ї	Њ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ
B0	°	±	І	і	Г	μ	¶	·	ё	№	е	»	ј	Ѕ	Ѕ	і
C0	A	B	B	Г	Д	Е	Ж	З	И	Й	К	Л	М	Н	О	П
D0	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я
E0	а	б	в	г	д	е	ж	з	и	й	к	л	м	н	о	п
F0	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я

Рисунок 1.1 – Алфавит входных символов

Фундаментом языка программирования SAA-2025 является данная кодировка.

1.5 Типы данных

В языке SAA-2025 программирования реализованы 4 типа данных: целый беззнаковый, символьный, строковый и логический. Описание типов данных, предусмотренных в данном языке, представлено в таблице 1.2.

Таблица 1.2 – Типы данных языка SAA-2025

Тип данных	Описание типа данных
Целочисленный беззнаковый тип данных <code>ubyte</code>	Используется для работы с целочисленными положительными значениями. В памяти занимает 1 байт. Диапазон значений: от 0 до 255. Инициализация по умолчанию: значение 0. Поддерживает все операции.
Строковый тип данных <code>string</code>	Используется для работы со строками, которые состоят из символьного типа <code>char</code> . Инициализация по умолчанию: символ конца строки «\0».
Символьный тип данных <code>char</code>	Используется для работы с символом, который в памяти занимает 1 байт. Инициализация по умолчанию: символ конца строки «\0».
Логический тип данных <code>bool</code>	Используется для работы с логическими значениями. В памяти занимает 1 байт. Принимает значения 0 и 1. Инициализация по умолчанию: 0.

Создание и использование пользовательских типов данных не предусмотрено.

1.6 Преобразование типов данных

Преобразование типов данных не поддерживается, так как язык является строго типизированным. Но в стандартной библиотеке есть функции преобразования числа в символ и наоборот.

1.7 Идентификаторы

Идентификатор языка программирования – это имя, задаваемое в программе для переменной и функции. В имени идентификатора допускаются только символы латинского алфавита и знак «`_`». Максимальная длина имени идентификатора – 10 символов. При вводе идентификатора длиной более

разрешенного количества символов, он будет усекается. Имя идентификатора не может совпадать с именем функции, уже содержащаяся в стандартной библиотеке, если только это функция подключена через оператор `add`.

1.8 Литералы

Литерал – это явное представление фиксированного значения в исходном коде программы. С помощью литералов осуществляется инициализация переменных. В языке существует четыре типа литералов. Краткое описание литералов языка SAA-2025 представлено в таблице 1.3.

Таблица 1.3 – Описание литералов

Тип литерала	Регулярное выражение	Описание	Пример
Целочисленный литерал	<code>[1-9]+[0-9]*</code>	Целочисленные неотрицательные литералы. Диапазон значений от 0 до 255	<code>declare ubyte chislo; chislo = 5;</code> 5 – целочисленный литерал 1 байт.
Логический Литерал	<code>[false true]</code>	Принимает значение 0 и 1. Используются для указания истинности	<code>declare bool log; log = true;</code> true – логический литерал
Символьный литерал	<code>'[a-zA-Z A-Я a-я 0-9 !-/]'</code>	Символ, заключенный в '' (одинарные кавычки),	<code>declare char symbol; symbol = 'q';</code> q – символьный литерал.
Строковый литерал	<code>"[a-zA-Z A-Я a-я 0-9 !-/]"</code>	Строковые литералы, максимальная длина строки 255 символов	<code>write "text"</code> text – строковый литерал

Численные литералы не требуют особого обозначения, логические определяются ключевыми словами «true» и «false», а символьные и строковые выделяются кавычками.

1.9 Объявление данных

Объявление переменных в языке SAA-2025 происходит помощью ключевого слова «declare». объявлении предусмотрена неявная инициализация. Явная инициализация при объявлении запрещена. Все переменные должны находиться внутри программного блока. Можно объявлять переменные с одинаковым идентификатором в различных функциях, потому что каждая переменная получает префикс – название функции, в которой она объявлена. Глобальные переменные не предусмотрены языком.

1.10 Инициализация данных

При объявлении переменной не допускается инициализация. При попытке инициализации при объявлении будет вызвана соответствующая ошибка на этапе синтаксического анализа. Описание способов инициализации переменных языка SAA-2025 представлено в таблице 1.4.

Таблица 1.4 – Способы инициализации переменных

Конструкция	Описание	Пример
declare <тип данных> <идентификатор>;	Неявная инициализация: переменные типа ubyte инициализируются нулем, переменные типа char и string – пустым символом, переменные типа bool – значением «0».	declare ubyte z; declare char ch; declare string str; declare bool logic;
<идентификатор> = <значение>;	Явное присваивание переменной значения.	z = 21; ch = 'W'; str = "Artyom"; logic = true;

Проверка на соответствие типов происходит на этапе семантического анализа.

1.11 Инструкции языка

Все возможные инструкции языка программирования SAA-2025 представлены в общем виде в таблице 1.5.

Таблица 1.5 – Инструкции языка программирования SAA-2025

Инструкция	Запись на языке SAA-2025
1	2
Объявление переменной	declare <тип данных> <идентификатор>;
Объявление функции	declare <тип данных> func <идентификатор> (<тип данных> <идентификатор>, ...) {<блок кода>;}
Присваивание	<идентификатор> = <значение>/<идентификатор>;

Продолжение таблицы 1.5

1	2
Объявление внешней функции	add <тип данных> func <идентификатор> (<тип данных> <идентификатор>, ...);
Блок инструкций	main { ... }
Возврат из подпрограммы	return <идентификатор> / <литерал>;
Циклическая инструкция	do[<блок кода>]while(<условие>;
Однострочный комментарий до конца строки	\$<любой текст>
Условная инструкция	if(<условие>)[<блок кода>;
Вывод данных	out <идентификатор> / <литерал>;

Данные инструкции представляют базовый набор средств для написания программы на языке SAA-2025.

1.12 Операции языка

Язык программирования SAA-2025 может выполнять операции, представленные в таблице 1.6.

Таблица 1.6 – Операции языка программирования SAA-2025

Операция	Примечание	Типы данных	Пример
1	2	3	4
()	Приоритет операций		sum = (a + b) * c;
=	Присваивание	(ubyte, ubyte) (char, char) (string, string) (bool, bool)	a = 15; ch = 'T'; str = "Me"; logic = true;
<, >	Знаки «больше», «меньше» для условной инструкции	(ubyte, ubyte) (char, char)	while(sum < diff) [...];
^	Оператор эквивалентности	(ubyte, ubyte) (char, char)	if(sum ^ diff) [...];
!	Оператор неравенства	(ubyte, ubyte) (char, char)	do [...] while(sum ! diff);
&	Логическое и	(ubyte, ubyte)	and = a & b;
	Логическое или	(ubyte, ubyte)	or = a b;
~	Инверсия	(ubyte)	inv = ~a;
+	Суммирование	(ubyte, ubyte) (char, char)	sum = a + b;

Продолжение таблицы 1.6

1	2	3	4
-	Вычитание	(ubyte, ubyte) (char, char)	diff = a – b;
*	Умножение	(ubyte, ubyte)	mul = a*b;
:	Деление	(ubyte, ubyte)	div = a:b;
%	Остаток от деления	(ubyte, ubyte)	mod = a % b;
/	Сдвиг влево	(ubyte, ubyte)	pr = a / b;
\	Сдвиг вправо	(ubyte, ubyte)	pr = a \ b;

Т.к. отрицательные числа не поддерживаются, если результат операции меньше нуля, он вычитается из максимального значения. Результат больше максимального значения прибавляется к нулевому значению.

1.13 Выражения и их вычисления

Выражение представляет собой комбинацию операторов и операндов. Круглые скобки в выражении используются для изменения приоритета операций. Также не допускается запись двух подряд идущих арифметических операций, за исключением операции побитовой инверсии, которая является унарной. Выражение может содержать вызов функции, если эта функция уже содержится в стандартной библиотеке. Выражения вычисляются только после оператора присваивания.

1.14 Конструкции языка

Ключевые программные конструкции языка программирования SAA-2025 представлены в таблице 1.7.

Таблица 1.7 – Программные конструкции языка SAA-2025

Конструкция	Запись на языке SAA-2025
Главная функция (точка входа)	main { ... return <идентификатор> / <литерал>; }
Функция	declare <тип данных> func <идентификатор> (<тип> <идентификатор>, ...) {... return <идентификатор> / <литерал>; };
Цикл	do[...]while(a^8);
Условный оператор	if(5>4)[...];

Все программные конструкции, кроме главной функции, заканчиваются символом «;».

1.15 Область видимости идентификаторов

В языке SAA-2025 все переменные являются локальными, т.е. имеют функциональную область видимости. Они обязаны находиться внутри программного блока функций. Каждая переменная получает префикс, который зависит от функции, в которой она находится. Объявление глобальных переменных не предусмотрено.

1.16 Семантические проверки

Таблица с перечнем семантических проверок, предусмотренных языком, приведена в таблице 1.8.

Таблица 1.8 – Семантические проверки

Номер	Правило
1	Идентификаторы не должны повторно объявляться в пределах одной функции.
2	Тип возвращаемого значения должен совпадать с типом функции при ее объявлении или подключении
3	Тип данных передаваемых значений в функцию должен совпадать с типом параметров при ее объявлении или подключении
4	В функцию должно быть передано то число параметров, сколько ожидается
5	Тип данных результата выражения должен совпадать с типом данных идентификатора, которому оно присваивается
6	Типы данных операндов выражения должны быть одинаковыми
7	Тип данных string не может быть аргументом условной конструкции
8	Для типа char определены только операции + и -
9	Функции не должны подключаться дважды в пределах одной программы
10	Для переменных типа bool использование любых операторов запрещено
11	Количество параметров функции не может быть больше 10

Если семантическая проверка проходит, то в лог журнал записывается соответствующая ошибка.

1.17 Распределение оперативной памяти на этапе выполнения

На этапе выполнения все переменные помещаются в кучу.

1.18 Стандартная библиотека и ее состав

Стандартная библиотека SAA-2025 написана на языке программирования C++. Функции стандартной библиотеки с описанием представлены в таблице 1.9.

Таблица 1.9 – Состав стандартной библиотеки

Функция(C++)	Возвращаемое значение	Описание
int ord(char ch)	char	Параметры: char ch – символ. Возвращает код символа
char* CurrentTime()	char*	Параметры: отсутствуют. Возвращает строку в формате ЧЧ:ММ.
char* CurrentDate()	char*	Параметры: отсутствуют. Возвращает строку в формате ДД.ММ.ГГГГ.
char chr(int number)	char	Параметры: int number – код символа. Возвращает символ с заданным кодом

Для использования функций стандартной библиотеки, нужно явно подключить необходимую функцию с помощью ключевого слова `add`, далее работа с ними производится как с пользовательскими функциями.

Так же в библиотеке присутствуют приватные функции. Их описание представлено в таблице 1.10.

Таблица 1.10 – Приватные функции стандартной библиотеки

Функция(C++)	Возвращаемое значение	Описание
void outputbyte (short a)	–	Выводит число на экран Вызывается оператором <code>write</code>
void outputchar (char a)	–	Выводит символ на экран Вызывается оператором <code>write</code>
void outputstring (void* in)	–	Выводит строку на экран Вызывается оператором <code>write</code>
void outputbool(bool a)	–	Выводит true или false на экран Вызывается оператором <code>write</code>

Приватные функции не могут быть вызваны явно и не требуют предварительного пользовательского подключения. Они вызываются специальными операторами языка.

1.19 Ввод и вывод данных

В языке SAA-2025 не реализованы средства ввода данных.

Для вывода данных в стандартный поток вывода предусмотрен оператор `write`, который базируется на приватных функциях стандартной библиотеки.

1.20 Точка входа

В языке SAA-2025 каждая программа должна содержать главную функцию `main`, т. е. точку входа, с которой начнется последовательное выполнение программы. Правило использования функции `main` было рассмотрено в таблице 1.5. При отсутствии точки входа или при определении нескольких, будут вызваны соответствующие исключения.

1.21 Препроцессор

Препроцессор в языке программирования SAA-2025 не предусмотрен.

1.22 Соглашения о вызовах

В языке вызов функций происходит по соглашению о вызовах `stdcall`. Особенности `stdcall`:

- все параметры функции передаются через стек;
- память высвобождает вызываемый код;
- занесение в стек параметров идет справа налево.

1.23 Объектный код

Язык программирования SAA-2025 транслируется в Ассемблер. Далее при вызове транслятора Ассемблера получается объектный код.

1.24 Классификация сообщений транслятора

В случае возникновения ошибки в коде программы на языке SAA-2025 и выявления ее транслятором в текущий файл протокола выводится сообщение. Классификация сообщений приведена в таблице 1.10.

Таблица 1.11. – Классификация сообщений транслятора

Интервал	Описание ошибок
0-99	Системные ошибки
100-109	Ошибки параметров
110-119	Ошибки открытия и чтения файлов
120-129	Ошибки лексического анализа
130-139	Ошибки таблиц лексем и таблиц идентификаторов
600-699	Ошибки синтаксического анализа
700-900	Ошибки семантического анализа

При помощи этих ошибок проще понять где именно произошла ошибка и по какой причине.

1.25 Контрольный пример

Код контрольного примера представлен в Приложении А.

2 Структура транслятора

2.1 Компоненты транслятора, их назначение и принципы взаимодействия

Транслятор преобразует программу, написанную на языке SAA-2025 в программу на языке ассемблера[1][2]. Для указания выходных файлов используются входные параметры транслятора, которые описаны в пункте 2.2. Компонентами транслятора являются лексический, синтаксический и семантический анализаторы, а также генератор кода на язык ассемблера. Принцип их взаимодействия представлен на рисунке 2.1.

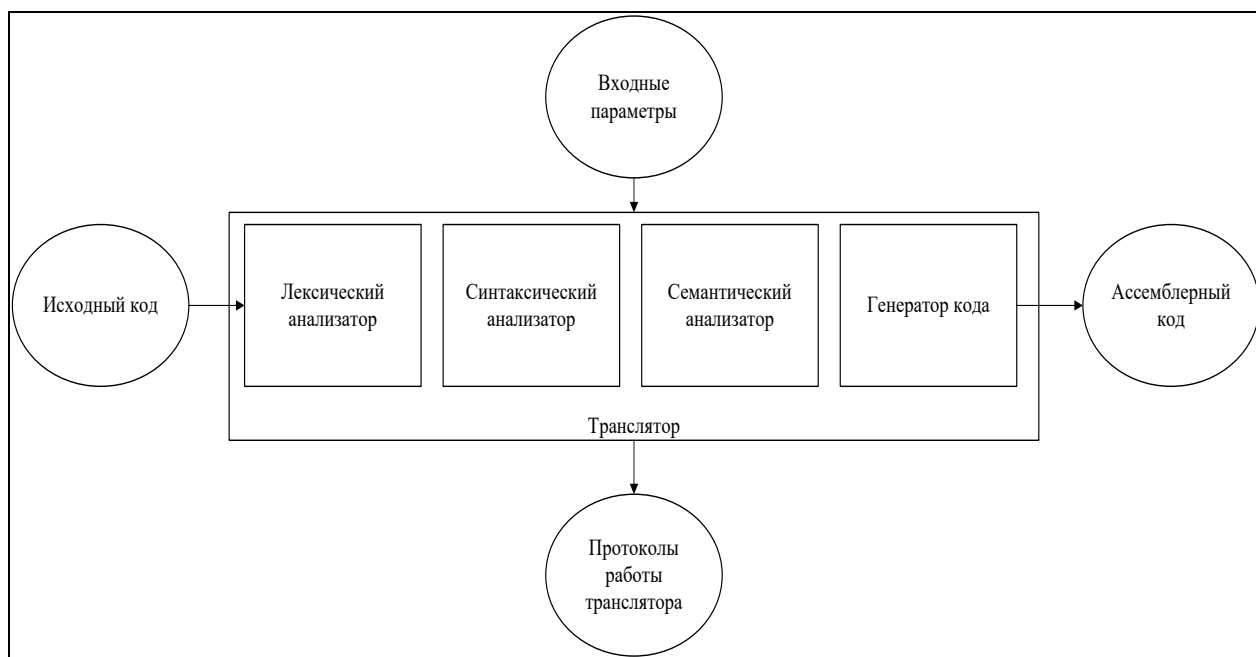


Рисунок 2.1 – Структура транслятора

Лексический анализ – первая фаза трансляции. Назначением лексического анализатора является нахождение ошибок лексики языка SAA-2025 и формирование таблицы лексем и таблицы идентификаторов.

Синтаксический анализ – это основная часть транслятора, предназначенная для распознавания синтаксических конструкций и формирования промежуточного кода SAA-2025. Для этого используются таблица лексем и идентификаторов. Синтаксический анализатор распознает синтаксические конструкции, выявляет синтаксические ошибки при их наличии и формирует дерево разбора.

Семантический анализ в свою очередь является проверкой исходной программы SAA-2025 на семантическую согласованность с определением языка.

Генератор кода – этап транслятора, выполняющий генерацию ассемблерного кода на основе полученных данных на предыдущих этапах трансляции. Генератор кода принимает на вход таблицы идентификаторов и

лексем и транслирует код на языке SAA-2025, прошедший успешно все предыдущие этапы, в код на языке ассемблера.

2.2 Перечень параметров транслятора

Входные параметры вводятся с помощью командной строки. Входные параметры транслятора представлены в таблице 2.1.

Таблица 2.1 – Входные параметры транслятора языка SAA-2025

Входной параметр	Описание	Значение по умолчанию
-in:<имя_файла>	Входной файл с любым расширением, в котором содержится исходный код на языке SAA-2025. Данный параметр должен быть указан обязательно. В случае если он не будет задан, то выполнение этапа трансляции не начнется.	Отсутствует
-log:<имя_файла>	Файл содержит в себе краткую информацию об исходном коде на языке SAA-2025. В этот файл могут быть выведены таблицы идентификаторов, лексем, а также дерево разбора.	<имя_файла>.log
-out:<имя_файла>	В этот файл будет записан результат трансляции кода на язык assembler	<имя_файла>.asm
tree	Вывод дерева разбора синтаксического анализатора.	false
lex	Вывод таблицы лексем	false
id	Вывод таблицы идентификаторов	false

Как видно из таблицы, транслятор языка SAA-2025 имеет 6 входных параметра. Если не указать параметры -log или -out, то по умолчанию они будут созданы с названием файла, указанного в параметре -in.

2.3 Перечень протоколов, формируемых транслятором

Таблица с перечнем протоколов, формируемых транслятором языка SAA-2025 и их назначением представлена в таблице 2.2.

Таблица 2.2 – Протоколы, формируемые транслятором языка SAA-2025

Формируемый протокол	Описание протокола
Файл журнала, «*.log»	В этот файл выводится протокол работы анализаторов, а так же различные ошибки
«*.asm»	Содержит сгенерированный код на языке Ассемблера.

В файл, с расширением «.log» выводится информация обо всех ошибках. Также при указании параметров «lex», «id» и «tree» туда записывается таблица лексем, таблица идентификаторов, шаги синтаксического анализатора и дерево разбора программы.

3 Разработка лексического анализатора

3.1 Структура лексического анализатора

Лексический анализатор – часть транслятора, выполняющая лексический анализ. Лексический анализатор принимает обработанный и разбитый на отдельные компоненты исходный код на языке SAA-2025. Структура лексического анализатора представлена на рисунке 3.1.

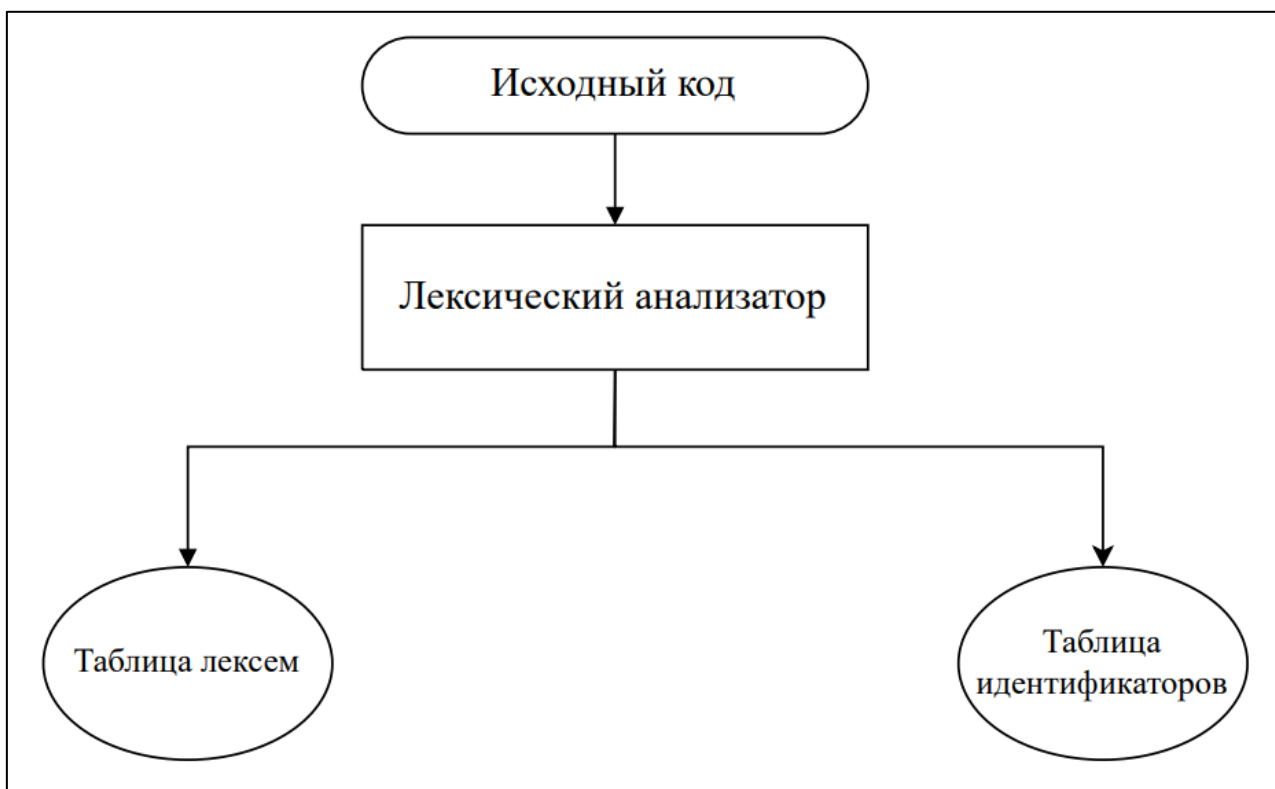


Рисунок 3.1 – Структура лексического анализатора

Результатом работы лексического анализатора являются заполненные таблица лексем и таблица идентификаторов.

3.2 Входные и выходные данные лексического анализатора

Входными данными для лексического анализатора является исходный код программы. Выходные данные лексического анализатора – это таблица лексем и таблица идентификаторов, которые используются на последующих этапах трансляции.

3.3 Параметры лексического анализатора

Входные параметры используются для вывода результата работы лексического анализатора. Они передаются аргументами через командную строку. Данные параметры были рассмотрены в таблице 2.1.

3.4 Алгоритм лексического анализа

Алгоритм работы лексического анализа заключается в последовательном распознавании и разборе цепочек исходного кода и заполнение таблиц идентификаторов и лексем. Лексический анализатор производит распознает и разбирает цепочки исходного текста программы. Это основывается на работе конечных автоматов, которую можно представить в виде графов. В случае, если подходящий автомат не был обнаружен, запоминается номер строки, в которой находился этот токен и выводится сообщение об ошибке. Если токен разобран, то дальнейшие действия, которые будут с ним производиться, будут зависеть от того, чем он является. Регулярные выражения — аналитический или формульный способ задания регулярных языков. Они состоят из констант и операторов, которые определяют множества строк и множество операций над ними. Любое регулярное выражение можно представить в виде графа.

В случае, если токен является идентификатором, перед его именем записывается название функции, в которой он объявлен и после этого он заносится в таблицу идентификаторов.

В случае, если токен является идентификатором функции, название функции в которой он объявлен не записывается.

В случае, если токен является литералом, то он заносится в таблицу идентификаторов с именем вида «ab_i», где a — имя функции, где объявлен литерал, b — «\$LEX», c — количество определенных литералов+1.

Когда встречаем токен, являющийся ключевым словом, которое отвечает за тип данных или вид идентификатора, заносим лексему, соответствующую ему, в таблицу лексем и запоминаем тип данных или вид идентификатора, которому он соответствует.

В последствии, когда встречаем идентификатор, заносим его в таблицу идентификаторов с соответствующим ему типом данных и видом идентификатора, и именем вида «ab», где a — имя функции, где объявлен идентификатор, b — имя идентификатора.

Конечный автомат, соответствующий ключевому слову «main» представлен в листинге 3.1.

```

FST l_main(
    str,
    5,
    NODE(1, RELATION('m', 1)),
    NODE(1, RELATION('a', 2)),
    NODE(1, RELATION('i', 3)),
    NODE(1, RELATION('n', 4)),
    NODE());

```

Листинг 3.1 – конечный автомат для ключевого слова «main»

В структуре «FST» «NODE» отвечает за узел графа, внутри этого узла указывается количество рёбер и сами рёбра, описанные структурой «RELATION», в этой структуре указывается символ, соответствующий переходу и номер узла, в который он ведёт.

Затем вызывается функция «execute», которая методом двух массивов разбирает цепочку в соответствии с этим конечным автоматом. Затем идёт реализация метода двух массивов, сначала создаётся массив и под него выделяется память, затем в цикле выполняются шаги, которые являются шагами алгоритма разбора конечного автомата. И в зависимости от результата разбора возвращается «true» - если разбор прошёл удачно и «false» - в ином случае. Реализация функции представлена на листинге 3.2.

```

bool FST::execute(FST& fst)
{
    fst.rstates[0] = 0;
    fst.position = -1;
    colum.reset_col();
    short* rstates = new short[fst.nstates];
    memset(rstates, 0xff, sizeof(short) * fst.nstates);
    short lstring = strlen(fst.string);
    bool rc = true;
    for (short i = 0; i < lstring && rc; i++)
    {
        fst.position++;
        rc = step(fst, rstates, lstring);
        colum.set_col(i);
    }
    delete[] rstates;
    return (rc ? (fst.rstates[fst.nstates - 1] == lstring) :
rc);
}

```

Листинг 3.2 – реализация функции «execute»

В случае успешного разбора цепочки, в зависимости от лексемы, соответствующей конечному автомату, выполняются различные действия, в случае ключевого слова «main» будет вызван код, указанный на листинге 3.3

```

case LEX_MAIN: {
    if (!wasMain) {
        isRequireBodyFunc = true;
        wasMain = true;
        break;}
    else
        throw ERROR_THROW_IN(125, lex.line, lex.col);}

```

Листинг 3.3 – код, вызванный для ключевого слова «char»

В результате программа получит информацию о наличии точки входа и при появлении повторной вызовет соответствующую ошибку.

3.5 Контроль входных символов

Исходный код на языке программирования SAA-2025 прежде чем транслироваться проверяется на допустимость символов. То есть изначально из входного файла считывается по одному символу и проверяется является ли он разрешенным.

Таблица для контроля входных символов представлена в приложении Б.

Принцип работы таблицы заключается в соответствии значения каждому элементу в шестнадцатеричной системе счисления значению в таблице ASCII.

Категории входных символов представлены в таблице 3.1.

Таблица 3.1 - Соответствие символов и их значений в таблице

Значение в таблице входных символов	Символы
Разрешенный символ	T
Запрещенный символ	F
Пробельный символ	S
Символ одинарной кавычки	C
Символ-разделитель или оператор	L
Символ двойной кавычки	D
Символ начала комментария	O
Символ новой строки	N

Как видно из таблицы, в языке достаточно много категорий символов, которые позволяют нам проще обрабатывать исходный текст программы.

3.6 Удаление избыточных символов

Удаление избыточных символов не предусмотрено, так как после проверки на допустимость символов исходный код на языке программирования SAA-2025 разбивается на токены, которые записываются в очередь.

3.7 Перечень ключевых слов.

Лексемы – это символы, соответствующие ключевым словам, символам операций и сепараторам, необходимые для упрощения дальнейшей обработки исходного кода программы. Данное соответствие описано в таблице 3.2.

Таблица 3.2 – Соответствие ключевых слов, символов операций и сепараторов с лексемами

Тип цепочки	Примечание	Цепочка	Лексема
1	2	3	4
Тип данных	Целочисленный беззнаковый тип данных	ubyte	t
	Логический тип данных	bool	t
	Строковый тип данных	string	t
	Символьный тип данных	char	t
Лексема	Объявление переменной	declare	d
	Подключение функции библиотеки	add	a
	Оператор вывода	out	o
	Объявление функции	func	f
	Возврат значения из функции	return	r
	Инструкция цикла	do-while	u
	Инструкция Условного оператора	if	o
	Блок инструкции цикла и условного оператора	[	[
		]	]
	Блок функции	{	{
		}	}
	Изменение приоритетности в выражении и отделение параметров функций	(	(
		)	)
	Сепараторы	;	;
		,	,
	Оператор присваивания	=	v
	Условный оператор	<	b
		>	b
		!	b
		^	b
Идентификатор		[a-z A-Z]+ [a-z A-Z 0-9]*	i
Оператор	Знаки арифметических и побитовых операций	+	v
		-	v
		*	v
		/	v
		\	v
		:	v
		%	v
		&	v
			v
		~	n

Продолжение таблицы 3.2

1	2	3	4
Литерал	Целочисленный литерал	[1-9]+[0-9]*	l
	Символьный литерал	[a-z A-Z 0-9]* кроме ‘	l
	Строковый литерал	[a-z A-Z 0-9]* кроме ”	l
Точка входа		main	m

Каждому выражению соответствует детерминированный конечный автомат, то есть автомат с конечным состоянием, по которому происходит разбор данного выражения. На каждый автомат в массиве подается фраза и с помощью регулярного выражения, соответствующего данному графу переходов, происходит разбор. В случае успешного разбора выражения оно записывается в таблицу лексем. Если выражение является идентификатором или литералом, информация также заносится в таблицу идентификаторов. Пример реализации таблицы лексем представлен в приложении В.

Также в приложении В находятся конечные автоматы, соответствующие лексемам языка SAA-2025.

3.8 Основные структуры данных

Основные структуры таблиц лексем и идентификаторов данных языка SAA-2025, используемых для хранения, представлены в приложении В. В таблице лексем содержится лексема, ее номер, полученный при разборе, номер строки в исходном коде, номер столбца в исходном коде, индекс таблицы идентификаторов (если нет соответствующего идентификатора, то индекс равен -1), а также специальное поле, в котором хранится значение лексемы. В таблице идентификаторов содержится имя идентификатора, номер в таблице лексем, тип данных, тип идентификатора, его значение, а также бинарное поле для определения внешний ли идентификатор.

3.9 Структура и перечень сообщений лексического анализатора

Индексы ошибок, обнаруживаемых лексическим анализатором, находятся в диапазоне 120-125. Перечень сообщений лексического анализатора представлен в таблице 3.3.

Таблица 3.3 – ошибки, определяемые синтаксическим анализатором

Номер ошибки	Сообщение об ошибке
1	2
120	[LA]: Ошибка при разборе токена

Продолжение таблицы 3.3

1	2
121	[LA]: Используется необъявленный идентификатор
122	[LA]: Идентификатор не имеет типа
124	[LA]: Отсутствует точка входа
125	[LA]: Обнаружена вторая точка входа

Также сам текст ошибки содержит в себе префикс [LA].

3.10 Принцип обработки ошибок

Все ошибки являются критическими и приводят к прекращению работы транслятора и выводу диагностического сообщения в log-файл.

3.11 Контрольный пример

Результат работы лексического анализатора – таблицы лексем и идентификаторов – представлен в приложении В.

4 Разработка синтаксического анализатора

4.1 Структура синтаксического анализатора

Синтаксический анализатор – это часть программы, которая выполняет процесс синтаксического анализа.

Синтаксический анализ – это фаза трансляции, выполняемая после лексического анализа и предназначенная для распознавания синтаксических конструкций SAA-2025. Структура синтаксического анализатора представлена на рисунке 4.1.

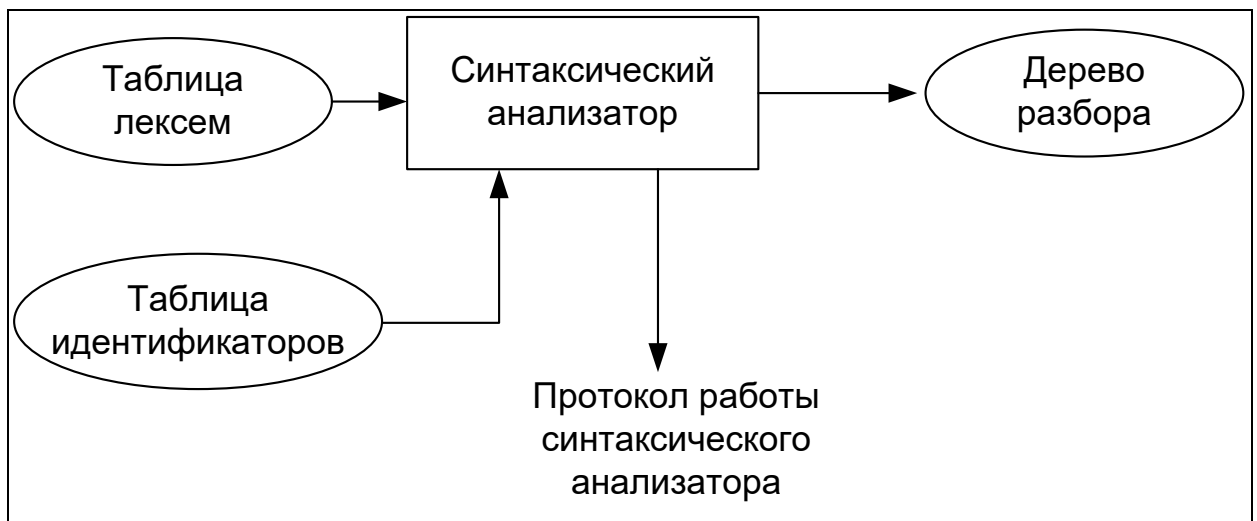


Рисунок 4.1 – Структура синтаксического анализатора SAA-2025

Входной информацией для синтаксического анализа является таблица лексем и таблица идентификаторов. Выходной информацией – дерево разбора и протокол работы анализатора, который описывает процесс анализа.

4.2 Контекстно свободная грамматика, описывающая синтаксис языка

В синтаксическом анализаторе транслятора языка SAA-2025 используется контекстно-свободная грамматика $G = \langle T, N, P, S \rangle$ [2], где

T – множество терминальных символов (было описано в разделе 1.2 данной пояснительной записки),

N – множество нетерминальных символов (первый столбец таблицы 4.1),

P – множество правил языка (второй столбец таблицы 4.1),

S – начальный символ грамматики, являющийся нетерминалом.

Эта грамматика имеет нормальную форму Грейбах, т.к. она не леворекурсивная (не содержит леворекурсивных правил) и правила P имеют вид:

1) $A \rightarrow a\alpha$, где $a \in T, \alpha \in (T \cup N) \cup \{\lambda\}$; (или $\alpha \in (T \cup N)^*$, или $\alpha \in V^*$)

2) $S \rightarrow \lambda$, где $S \in N$ - начальный символ, при этом если такое правило существует, то нетерминал S не встречается в правой части правил.

Грамматика языка SAA-2025 представлена в приложении Г.

TS – терминальные символы, которыми являются сепараторы, знаки арифметических операций и некоторые строчные буквы.

NS – нетерминальные символы, представленные несколькими заглавными буквами латинского алфавита.

Таблица 4.1 – Перечень правил и описание нетерминальных символов SAA-2025

Нетерминал	Цепочки правил	Описание
1	2	3
S	$m\{NrE;\};$ $m\{rE;\};$ $dtfi(F)\{NrE;\};S$ $dtfi(F)\{rE;\};S$ $dtfi()\{NrE;\};S$ $dtfi()\{rE;\};S$ $m\{rE;\};S$	Порождает правила, описывающее общую структуру программы
N	$dti;$ $rE;$ $ivE;$ $atfi(F);$ $o(B)[N]N$ $o(B)[N]$ $dti;N$ $ivE;N$ $etfi(F);N$ $wi;$ $wl;$ $wi;N$ $wl;N$ $u(B)[N];$ $u(B)[N];N$ $atfi();N$ $atfi();$	Порождает правила, описывающие конструкции языка
E	i l ni nl (E)	Порождает правила, описывающие выражения
E	$i(W)$ $i()$ iM lM niM nlM	Порождает правила, описывающие выражения

Продолжение таблицы 4.1

1	2	3
	n(E)M (E)M i(W)M n(E)	
F	ti ti,F i i,F l,F	Порождает правила, описывающие параметры локальной функции при ее объявлении
W	i l i,W l,W	Порождает правила, описывающие принимаемые параметры функции
B	ibi ibl lbi lbl i	Порождает правила, описывающие условное выражение в операторе цикла и условном операторе
M	vE v(E) v(E)M vEM	Порождает правила, описывающие знаки арифметических операций

Протокол и ошибки работы синтаксического анализатора выводятся в «log» журнал.

4.3 Построение конечного магазинного автомата

Конечный автомат с магазинной памятью представляет собой семерку $M = \langle Q, V, Z, \delta, q_0, z_0, F \rangle$ [1], описание которой представлено в таблице 4.2. Структура данного автомата показана в приложении Г.

Таблица 4.2 – Описание компонентов магазинного автомата

Компонента	Определение	Описание
1	2	3
Q	Множество состояний автомата	Состояние автомата представляет из себя структуру, содержащую позицию на входной ленте, номера текущего правила и цепочки и стек автомата
V	Алфавит входных символов	Алфавит является множеством терминальных и нетерминальных символов, описание которых содержится в разделе 1.2 и в таблице 4.1.
Z	Алфавит специальных магазинных символов	Алфавит магазинных символов содержит стартовый символ и маркер дна стека

Продолжение таблицы 4.2

1	2	3
δ	Функция переходов автомата	Функция представляет из себя множество правил грамматики, описанных в таблице 4.1.
q_0	Начальное состояние автомата	Состояние, которое приобретает автомат в начале своей работы. Представляется в виде стартового правила грамматики (нетерминальный символ A)
z_0	Начальное состояние магазина автомата	Символ маркера дна стека (\$)
F	Множество конечных состояний	Конечные состояние заставляют автомат прекратить свою работу. Конечным состоянием является пустой магазин автомата и совпадение позиции на входной ленте автомата с размером ленты

Для вывода результата работы синтаксического анализатора нужно использовать флаг tree.

4.4 Основные структуры данных

Основные структуры данных синтаксического анализатора включают в себя структуру магазинного конечного автомата и структуру грамматики Грейбах, описывающей правила языка SAA-2025. Данные структуры представлены в приложении Г.

4.5 Описание алгоритма синтаксического разбора

Принцип работы автомата следующий:

- в магазин записывается стартовый символ;
- на основе полученных ранее таблиц формируется входная лента;
- запускается автомат;
- выбирается цепочка, соответствующая нетерминальному символу, записывается в магазин в обратном порядке;
- если терминалы в стеке и в ленте совпадают, то данный терминал удаляется из ленты и стека. Иначе возвращаемся в предыдущее сохраненное состояние и выбираем другую цепочку нетерминала;
- если в магазине встретился нетерминал, переходим к пункту 4;
- если наш символ достиг дна стека, и лента в этот момент пуста, то синтаксический анализ выполнен успешно и формируется дерево разбора. Иначе генерируется исключение.

4.6 Параметры синтаксического анализатора

Для вывода результата работы синтаксического анализатора используются входные параметры, описанные в пункте 2.2. Перечень входных параметров транслятора в таблице 2.1.

4.7 Структура и перечень сообщений синтаксического анализатора

Индексы ошибок, обнаруживаемых синтаксическим анализатором, находятся в диапазоне 600-609. Перечень сообщений синтаксического анализатора представлен в таблице 4.3.

Таблица 4.3 – ошибки, определяемые синтаксическим анализатором

Номер ошибки	Сообщение об ошибке
600	[Syntaxis]: Неверная структура программы
601	[Syntaxis]: Ошибочный оператор
602	[Syntaxis]: Ошибка в выражении
603	[Syntaxis]: Ошибка в параметрах функции
604	[Syntaxis]: Ошибка в параметрах вызываемой функции
605	[Syntaxis]: Ошибка знака в выражении
606	[Syntaxis]: Ошибка синтаксического анализа
607	[Syntaxis]: Ошибка условной конструкции
609	[Syntaxis]: Обнаружена синтаксическая ошибка (смотри журнал Log)

Текст синтаксической ошибки содержит в себе префикс [Syntaxis].

4.8 Принцип обработки ошибок

Обработка ошибок происходит следующим образом:

- Синтаксический анализатор перебирает все правила и цепочки правила грамматики для нахождения подходящего соответствия с конструкцией, представленной в таблице лексем.

- Если невозможно подобрать подходящую цепочку, то генерируется соответствующая ошибка.

– В случае ошибки выводится соответствующее сообщение в журнал «log» и компилятор прекращает работу.

4.9 Контрольный пример

Пример разбора синтаксическим анализатором исходного кода на языке SAA-2025 представлен в приложении Д. Дерево разбора исходного кода также представлено в приложении Д.

5 Разработка семантического анализатора

5.1 Структура семантического анализатора

Семантический анализ происходит при выполнении фазы лексического анализа и реализуется в виде отдельных проверок текущих ситуаций в конкретных случаях: установки флага или нахождения в особом месте программы (оператор выхода из функции, оператор ветвления, вызов функции стандартной библиотеки). Структура семантического анализатора представлена на рисунке 5.1.

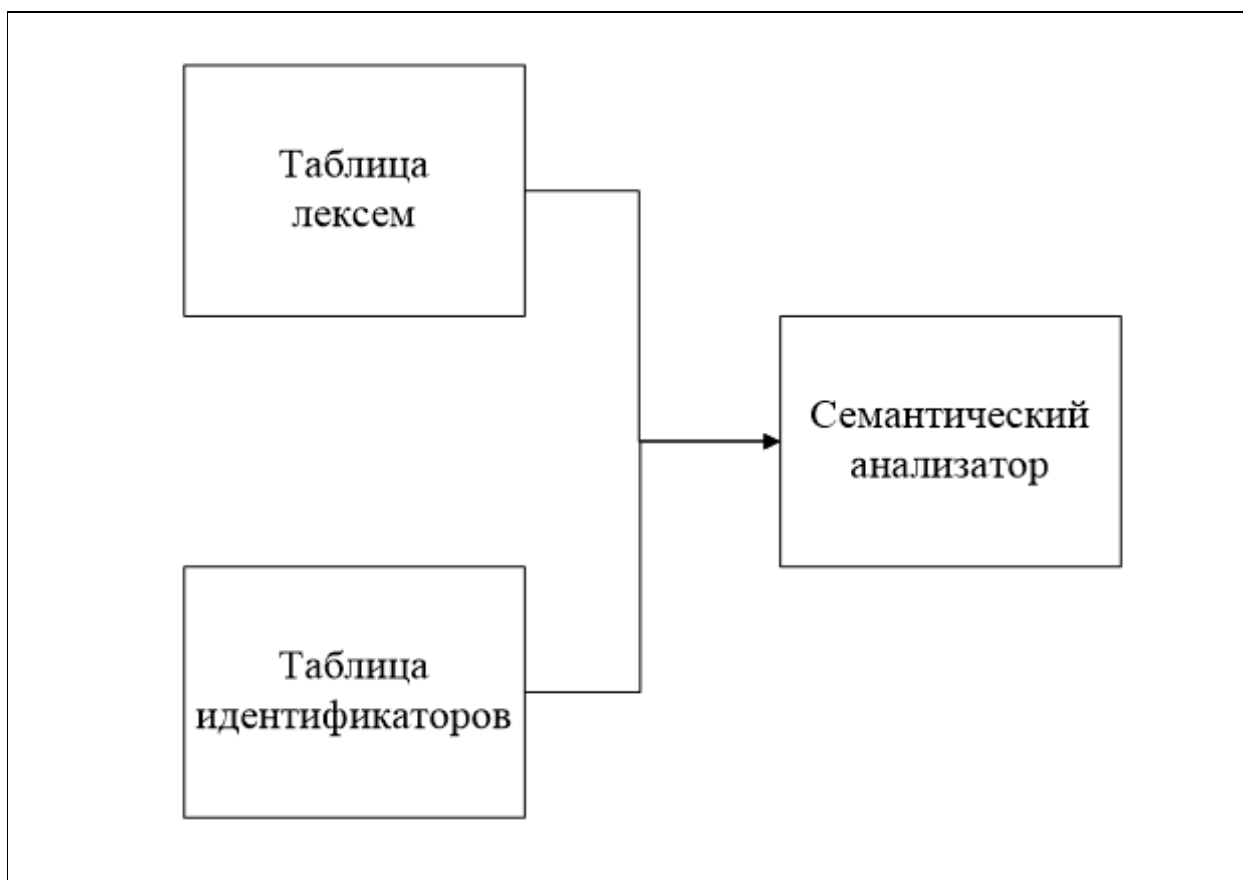


Рисунок 5.1 – Структура семантического анализатора

Функции семантического анализатора частично реализованы в лексическом анализаторе.

5.2 Функции семантического анализатора

Семантический анализатор выполняет проверку на основные правила языка (семантики языка), которые описаны в разделе 1.16.

5.3 Структура и перечень сообщений семантического анализатора

Все ошибки семантического анализатора имеют идентификатор свыше 700. Сообщения, формируемые семантическим анализатором, представлены в таблице 5.1.

Таблица 5.1 – ошибки, определяемые семантическим анализатором

Номер ошибки	Сообщение об ошибке
1	2
700	[Semantic]: Повторное объявление идентификатора
701	[Semantic]: Ошибка в возвращаемом значении
702	[Semantic]: Ошибка в передаваемых значениях в функции: количество параметров не совпадает
703	[Semantic]: Ошибка в передаваемых значениях в функции: типы параметров не совпадают
704	[Semantic]: Нарушены типы данных в выражении
705	[Semantic]: Ошибка экспорта: в библиотеке нет такой функции
706	[Semantic]: Ошибка экспорта: неверные параметры
707	[Semantic]: Ошибка экспорта: ошибочный тип возвращаемого значения
708	[Semantic]: Для строк операторы запрещены
709	[Semantic]: Ошибочные параметры условной конструкции: строки не могут быть параметрами условной конструкции
710	[Semantic]: Ошибочный оператор: для типа char разрешены только операции + и -

Продолжение таблицы 5.1

1	2
711	[Semantic]: Для логических переменных использование любых операторов запрещено
712	[Semantic]: Количество параметров не может быть больше 10

Текст семантической ошибки содержит в себе префикс [Semantic].

5.4 Принцип обработки ошибок

При обнаружении хотя бы одной ошибки транслятор завершит свою работу с записью информации об ошибке в «log» файл.

5.5 Контрольный пример

Результат работы контрольного примера расположен в приложении Ж, где показан результат лексического анализатора, т.к. представленные таблицы лексем и идентификаторов проходят лексическую и семантическую проверки одновременно.

6 Преобразование выражений

6.1 Выражения, допускаемые языком

В языке SAA-2025 допускаются выражения, применимые к целочисленным типам данных. В выражениях поддерживаются арифметические операции, такие как $+$, $-$, $*$, $\wedge$, $:$, $/$, $\backslash$, $\%$, $\sim$, $\&$, $|$ и $()$, и вызовы функций как операнды арифметических выражений.

Приоритет операций представлен в таблице 6.1.

Таблица 6.1 – Приоритет операций в языке SAA-2025

Операция	Значение приоритета
(	1
)	1
+	2
-	2
*	3
:	3
%	3
/	3
\	3
&	3
	3
~	4

Как видно из вышеизложенного, все операции имеют определенный приоритет, который указывает порядок выполнения операций.

6.2 Польская запись и принцип ее построения

Выражения в языке SAA-2025 преобразовываются к обратной польской записи.

Польская запись – это альтернативный способ записи арифметических выражений, преимущество которого состоит в отсутствии скобок.

Обратная польская запись – это форма записи математических и логических выражений, в которой операнды расположены перед знаками операций.

Алгоритм построения:

- читаем очередной символ;
- если он является идентификатором или литералом, то добавляем его к выходной строке;
- если символ является символом функции, то помещаем его в стек;
- если символ является открывающей скобкой, то она помещается в стек;
- исходная строка просматривается слева направо;

- если символ является закрывающей скобкой, то выталкиваем из стека в выходную строку все символы пока не встретим открывающую скобку. При этом обе скобки удаляются и не попадают в выходную строку;
- как только входная лента закончится все символы из стека выталкиваются в строку;
- в случае если встречаются операции, то выталкиваем из стека в выходную строку все операции, которые имеют выше приоритетность чем последняя операция;
- также, если идентификатор является именем функции, то он заменяется на спецсимвол «@».

Таблица 6.2 – Пример преобразования выражения в обратную польскую запись

Исходная строка	Результирующая строка	Стек
a+~255*(b 32)		
+~255*(b 32)	a	
~255*(b 32)	a	+
255*(b 32)	a	+~
*(b 32)	a255	+~
(b 32)	a255~	+*
b 32)	a255~	+*(
32)	a255~b	+*(
32)	a255~b	+*(
)	a255~b32	+*(
	a255~b32	+*
	a255~b32 *	+
	a255~b32 *+	

Как результат успешного разбора, мы получаем пустой стек и заполненную результирующую строку.

6.3 Программная реализация обработки выражений

Программная реализация алгоритма преобразования выражений к польской записи представлена в приложении Е.

6.4 Контрольный пример

Пример преобразования выражения к польской записи представлен в таблице 6.2. Преобразование выражений в формат польской записи необходимо для построения более простых алгоритмов их вычисления.

7 Генерация кода

7.1 Структура генератора кода

Генерация кода языка SAA-2025 осуществляется в язык Ассемблера Microsoft Macro Assembler (MASM), который используется для процессоров семейства x86.

Генерация кода осуществляется модулем «CodeGeneration». Вызывается функция «Generate». Эта функция вызывает 4 функции, которые и производят генерацию кода.

Первой вызывается функция «Head». Функция «Head» записывает в выходной файл инструкции, содержащие информацию о подключаемых к программе библиотекам, наборе инструкций и прототипах библиотечных функций и функции завершения программы.

После функции «Head» выполняется функция «Const», которая в сегмент констант выводит литералы и их значение. В комментариях после литерала указан соответствующий тип.

Следующей функцией является «Data», которая, аналогично функции «Constants», выводит в сегмент переменных переменные и инициализирует их начальным значением. После переменной в комментарии указывается её тип.

Последней из этих функций является функция «Code», которая отвечает за содержимое сегмента кода. В ней объявляются пользовательские функции и описывается их содержимое. Затем наступает очередь функции «main», в которой происходят те же действия. При определении пользовательской функции к ней добавляется префикс «\$», это необходимо для того, чтобы пользователь не мог создать конфликт со встроенными функциями.

Структура генератора кода представлена на листинге 7.1.

```
void GenerateCode(LEX::LEX lex, Out::OUT out) {  
    Head(out);  
    Const(out, lex);  
    Data(out, lex);  
    Code(out, lex);  
}
```

Листинг 7.1 – структура генератора кода

Как видно из листинга, в функции «GenerateCode» последовательно запускаются все остальные функции.

7.2 Представление типов данных в оперативной памяти

Моделью памяти сгенерированного кода является flat, так как транслятор является 32-разрядным.

Соответствия между типами данных идентификаторов на языке SAA-2025 и на языке ассемблера приведены в таблице 7.1.

Таблица 7.1 – Соответствие типов данных языка SAA-2025 и Ассемблера

Тип идентификатора на языке SAA-2025	Тип идентификатора на языке Ассемблера
char	BYTE
string	DWORD
ubyte	BYTE
bool	BYTE
Литерал char	BYTE
Литерал bool	BYTE
Литерал string	BYTE
Литерал ubyte	BYTE

Тип данных char может содержать символы таблицы кодировки «Windows-1251», которые находятся в диапазоне от «0» до «255», эти значения занимают 1 байт.

Тип данных bool может содержать логические значения «0» или «1». Для этого достаточно 1 байта.

Тип данных string содержит указатель на начало строки, указатель в модели памяти flat занимает 4 байта.

Тип данных ubyte принимает значения от «0» до «255». Для такого диапазона значений необходим 1 байт памяти.

7.3 Статическая библиотека

Статическая библиотека реализована на языке программирования C++. Ее код находится в проекте SAA-2025LIB, в свойствах которого был выбран пункт «статическая библиотека .lib». Состав статической библиотеки представлен в таблице 7.2.

Таблица 7.2 – состав статической библиотеки языка SAA-2025

Функция	Описание
1	2
outputstr	Получает на вход указатель на начало строки и выводит его
outputubyte	Получает на вход целое число и выводит
outputchar	Получает на вход символ и выводит его

Продолжение таблицы 7.2

1	2
outputbool	Получает на вход логическое значение и выводит его
GetTime	Возвращает указатель на начало строки, содержащую текущее время в формате ЧЧ:ММ
GetDate	Возвращает указатель на начало строки, содержащую текущую дату в формате ДД.ММ.ГГГГ
ord	Получает на вход символ и возвращает число, соответствующее кодировке символа
chr	Получает на вход число и возвращает символ, соответствующий кодировке этого числа.

Подключение библиотеки в языке ассемблера происходит с помощью директивы `includelib` на этапе генерации кода. Далее объявляются прототипы функций из библиотеки.

7.4 Особенности алгоритма генерации кода

Описание функций генерации кода было представлено в разделе 7.1.

Особенностями алгоритма генерации кода является следующее:

- в префикс функций добавляется символ «\$», чтобы не было конфликтов со встроенными функциями.
- присвоение значений происходит при помощи стека.
- каждой переменной в комментарии указывается её тип данных.
- перед выводом выражения в комментарии выводится её аналог из таблицы лексем, после преобразования в польскую обратную запись.
- в начале функции `main` подключаются функции для вывода переменных или литералов.

Обобщенная блок-схема алгоритма генерации кода представлена в приложении 3.

Исходя из блок-схемы, алгоритм генерации исходного кода сводится к реализации выражений, написанных на языке SAA-2025, на языке Ассемблера. Код производит проверку содержимого в выражении и, в зависимости от содержимого, выполняет определенные действия.

Перечень функций, используемых для реализации алгоритма генерации кода представлен в таблице 7.3.

Таблица 7.3 – перечень функций, используемых для генерации кода

Функция	Описание
Generate	Определяет начало этапа генерации кода
Head	Выводит заголовочную информацию
Constants	Выводит литералы в сегмент констант
Data	Выводит переменные с сегмент данных
Code	Начинает этап генерации сегмента кода, также ответственен за
WriteFunctions	Выводит информацию о функциях, их входные параметры,
InvokeExpressions	Переводит выражения в язык Ассемблера

Исходя из информации из таблицы, этап генерации кода содержит в себе множество функций, каждая из которых выполняет определенную задачу.

7.5 Параметры, управляющие генерацией кода

Генерируемый код записывается в файл заданный параметром «-out».

Если параметр «out» не был указан в параметрах командной строки, то по умолчанию выходной файл получит название файла, указанного с параметром «-in», который является обязательным.

7.6 Контрольный пример

Сгенерированный код можно посмотреть в приложении Ж.

8 Тестирование транслятора

8.1 Тестирование фазы проверки на допустимость символов

В основе тестов, которым подвергался транслятор, лежали использование недопустимых структур и выражений, попытка ввода недопустимых символов, использование заведомо приводящих к ошибке ситуаций.

При обнаружении компилятор прекращает свою работу и выводит в файл протокола соответствующее сообщение. Также вся информация, выведенная до встречи ошибки остаётся в файле протокола, чтобы можно было проследить за тем, что привело к ошибке.

8.2 Результаты тестирования

На этапе лексического анализа могут возникнуть ошибки, описанные в пункте 3.7. Результаты тестирования лексического анализатора показаны в таблице 8.1.

Таблица 8.1 – Результаты тестирования транслятора

Исходный код	Диагностическое сообщение
1	2
main{ declare string ?str; }	Ошибка 111: [IN]: Недопустимый символ в исходном файле (-in), строка 1, столбец 16
main{ s = 5; }	Ошибка 121: [LA]: Используется необъявленный идентификатор, строка 2, столбец 1
main{ string declare a; }	Ошибка 609: [Syntaxis]: Обнаружена синтаксическая ошибка(смотри журнал Log)
main{ declare char a; declare string a; }	Ошибка 700: [Semantic]: Повторное объявление идентификатора, строка 3, столбец 16
declare ubyte func ret_ubyte(ubyte q){ declare string str; return str; }; main{ declare ubyte num; num = ret_ubyte(3); return 0; }	Ошибка 701: [Semantic]: Ошибка в возвращаемом значении, строка 3, столбец 8

Продолжение таблицы 8.1

1	2
<pre>declare ubyte func ret_ubyte(ubyte q, ubyte w){ return q; }; main{ declare ubyte num; num = ret_ubyte(3); return 0; }</pre>	Ошибка 702: [Semantic]: Ошибка в передаваемых значениях в функции: количество параметров не совпадает, строка 6, столбец 18
<pre>declare string func ret_string(ubyte q){ declare string str; return str; }; main{ declare string str; str = ret_string(str); return 0; }</pre>	Ошибка 703: [Semantic]: Ошибка в передаваемых значениях в функции: типы параметров не совпадают, строка 7, столбец 18
<pre>declare ubyte func ret_ubyte(ubyte q){ declare string str; return str; }; main{ declare string str; str = ret_ubyte(str); return 0; }</pre>	Ошибка 704: [Semantic]: Нарушены типы данных в выражении, строка 7, столбец 7
<pre>main{ add ubyte func random(ubyte max); return 0; }</pre>	Ошибка 705: [Semantic]: Ошибка экспорта: в библиотеке нет такой функции, строка 2, столбец 16

Как видно из таблицы, проверки проводились на различные ситуации. Были исправлены все ошибки, которые происходили во время работы транслятора.

Заключение

В ходе выполнения курсовой работы был разработан транслятор и генератор кода для языка программирования SAA-2025 со всеми необходимыми компонентами. Таким образом, были выполнены основные задачи данной курсовой работы:

Сформулирована спецификация языка SAA-2025;

- Разработаны конечные автоматы и важные алгоритмы на их основе для эффективной работы лексического анализатора;

- Осуществлена программная реализация лексического анализатора, распознающего допустимые цепочки спроектированного языка;

- Разработана контекстно-свободная, приведённая к нормальной форме Грейбах, грамматика для описания синтаксически верных конструкций языка;

- Осуществлена программная реализация синтаксического анализатора;

- Разработан семантический анализатор, осуществляющий проверку используемых инструкций на соответствие логическим правилам;

- Разработан транслятор кода на язык ассемблера;

- Проведено тестирование всех вышеперечисленных компонентов.

Окончательная версия языка SAA-2025 включает:

- 4 типа данных;

- Поддержка оператора вывода;

- Возможность вызова 4 функций стандартной библиотеки;

- Наличие 7 арифметических операторов, 3 побитовых операторов, 4 операторов сравнения и скобок.

- Поддержка функций, операторов цикла и условия;

Проделанная работа позволила получить необходимое представление о структурах и процессах, использующихся при построении трансляторов, а также основные различия и преимущества тех или иных средств трансляции.

Список использованных источников

1. Наркевич А. С. Курс «Конструирование программного обеспечения»
2. Карпов Ю. Теория и технология программирования. Основы построения трансляторов, 2005. – 272с.
3. Введение в теорию трансляторов [Электронный ресурс]. – Режим доступа: <http://bourabai.ru/troi/compiler.htm>. – Дата доступа: 15.11.2025.
4. Википедия: Обратная польская запись [Электронный ресурс]. – Режим доступа: https://en.wikipedia.org/wiki/Reverse_Polish_notation. – Дата доступа: 20.11.2025.
5. MASM для x86 [Электронный ресурс]. – Режим доступа: <https://docs.microsoft.com/en-us/cpp/assembler/masm/masm-for-x64-m164-exe?view=msvc-160>. – Дата доступа: 29.11.2025.
6. Ирвин К. Р. Язык ассемблера для процессоров Intel / К. Р. Ирвин. – М.: Вильямс, 2005. – 912с.

ПРИЛОЖЕНИЕ А

```

$Вычисление факториала
declare ubyte func CalculateFactorial(ubyte number) {
    declare ubyte result;
    result = 1;
    do
    [
        result = result * number;
        number = number - 1;
        out number;
    ] while (number > 1);
    return result;
};

main
{
    out "-----";
    out "Factorial:";
    declare ubyte result;
    result = CalculateFactorial(3) + CalculateFactorial(4);
    out "!3 + !4";
    out result;

    declare bool bValue;
    bValue = true;
    if (bValue)
    [
        out "-----";
        out "bitwise operations in condition:";
        result = ((66 | 11) + 72) & (80 : (1 - 1));
        out "((66 | 11) + 72) & (80 : (2 - 1)) = ";
        out result;
    ];

    out "-----";
    out "Char operations:";
    declare char c;
    c = 'Z' - 'z';
    out "'Z' - 'z':" ;
    out c;
    c = 'a' + 'b';
    out "'a' + 'b':" ;
    out c;
    bValue = false;
    if (bValue)
    [
        c = 'A' + 'B';
        out "'A' + 'B':" ;
        out c;
    ];

    out "-----";

```

```
add string func CurrentDate();  
add string func CurrentTime();  
declare string dateNow;  
declare string timeNow;  
dateNow = CurrentDate();  
timeNow = CurrentTime();  
out "date:";  
out dateNow;  
out "time:";  
out timeNow;  
return 0;  
}
```

Листинг 1 – контрольный пример

Листинг 1 – Таблица входных символов

ПРИЛОЖЕНИЕ В

Позиция	№ строки	№ столбца	Лексема	Индекс ТИ
0	2	1	d	-1
1	2	9	t	-1
2	2	15	f	-1
3	2	20	i	0
4	2	38	(	-1
5	2	39	t	-1
6	2	45	i	1
7	2	51	)	-1
8	2	53	{	-1
9	3	2	d	-1
10	3	10	t	-1
11	3	16	i	2
12	3	22	;	-1
13	4	2	i	2
14	4	9	v	-1
15	4	11	l	3
16	4	12	;	-1
17	5	2	D	-1
18	6	2	[	-1
19	7	3	i	2
20	7	10	v	-1
21	7	12	i	2
22	7	19	v	-1
23	7	21	i	1
24	7	27	;	-1
25	8	3	i	1
26	8	10	v	-1
27	8	12	i	1
28	8	19	v	-1
29	8	21	l	3
30	8	22	;	-1
31	9	3	O	-1
32	9	7	i	1
33	9	13	;	-1
34	10	2	]	-1
35	10	4	u	-1
36	10	10	(	-1
37	10	11	i	1
38	10	18	b	-1
39	10	20	l	3
40	10	21	)	-1
41	10	22	;	-1
42	11	2	r	-1
43	11	9	i	2
44	11	15	;	-1
45	12	1	}	-1
46	12	2	;	-1
47	13	1	m	-1
48	14	1	{	-1
49	15	2	O	-1

50	15	14	l	4
51	15	15	;	-1
52	16	2	O	-1
53	16	18	l	5
54	16	19	;	-1
55	17	2	d	-1
56	17	10	t	-1
57	17	16	i	6
58	17	22	;	-1
59	18	2	i	6
60	18	9	v	-1
61	18	11	i	0
62	18	29	(	-1
63	18	30	l	7
64	18	31	)	-1
65	18	33	v	-1
66	18	35	i	0
67	18	53	(	-1
68	18	54	l	8
69	18	55	)	-1
70	18	56	;	-1
71	19	2	O	-1
72	19	15	l	9
73	19	16	;	-1
74	20	2	O	-1
75	20	6	i	6
76	20	12	;	-1
77	22	2	d	-1
78	22	10	t	-1
79	22	15	i	10
80	22	21	;	-1
81	23	2	i	10
82	23	9	v	-1
83	23	11	l	11
84	23	15	;	-1
85	24	2	o	-1
86	24	5	(	-1
87	24	6	i	10
88	24	12	)	-1
89	25	2	[	-1
90	26	3	O	-1
91	26	15	l	4
92	26	16	;	-1
93	27	3	O	-1
94	27	41	l	12
95	27	42	;	-1
96	28	3	i	6
97	28	10	v	-1
98	28	12	(	-1
99	28	13	(	-1
100	28	14	l	13
101	28	17	v	-1
102	28	19	l	14

103	28	21	)	-1
104	28	23	v	-1
105	28	25	l	15
106	28	27	)	-1
107	28	29	v	-1
108	28	31	(	-1
109	28	32	l	16
110	28	35	v	-1
111	28	37	(	-1
112	28	38	l	17
113	28	40	v	-1
114	28	42	l	3
115	28	43	)	-1
116	28	44	)	-1
117	28	45	;	-1
118	29	3	O	-1
119	29	45	l	18
120	29	46	;	-1
121	30	3	O	-1
122	30	7	i	6
123	30	13	;	-1
124	31	2	]	-1
125	31	3	;	-1
126	33	2	O	-1
127	33	14	l	4
128	33	15	;	-1
129	34	2	O	-1
130	34	24	l	19
131	34	25	;	-1
132	35	2	d	-1
133	35	10	t	-1
134	35	15	i	20
135	35	16	;	-1
136	36	2	i	20
137	36	4	v	-1
138	36	8	l	21
139	36	10	v	-1
140	36	14	l	22
141	36	15	;	-1
142	37	2	O	-1
143	37	18	l	23
144	37	19	;	-1
145	38	2	O	-1
146	38	6	i	20
147	38	7	;	-1
148	39	2	i	20
149	39	4	v	-1
150	39	8	l	24
151	39	10	v	-1
152	39	14	l	25
153	39	15	;	-1
154	40	2	O	-1
155	40	18	l	26

156	40	19	;	-1
157	41	2	O	-1
158	41	6	i	20
159	41	7	;	-1
160	42	2	i	10
161	42	9	v	-1
162	42	11	l	27
163	42	16	;	-1
164	43	2	o	-1
165	43	5	(	-1
166	43	6	i	10
167	43	12	)	-1
168	44	2	[	-1
169	45	3	i	20
170	45	5	v	-1
171	45	9	l	28
172	45	11	v	-1
173	45	15	l	29
174	45	16	;	-1
175	46	3	O	-1
176	46	19	l	30
177	46	20	;	-1
178	47	3	O	-1
179	47	7	i	20
180	47	8	;	-1
181	48	2	]	-1
182	48	3	;	-1
183	50	2	O	-1
184	50	14	l	4
185	50	15	;	-1
186	51	2	a	-1
187	51	6	t	-1
188	51	13	f	-1
189	51	18	i	31
190	51	25	(	-1
191	51	26	)	-1
192	51	27	;	-1
193	52	2	a	-1
194	52	6	t	-1
195	52	13	f	-1
196	52	18	i	32
197	52	25	(	-1
198	52	26	)	-1
199	52	27	;	-1
200	53	2	d	-1
201	53	10	t	-1
202	53	17	i	33
203	53	24	;	-1
204	54	2	d	-1
205	54	10	t	-1
206	54	17	i	34
207	54	24	;	-1
208	55	2	i	33

209	55	10	v	-1
210	55	12	i	31
211	55	19	(	-1
212	55	20	)	-1
213	55	21	;	-1
214	56	2	i	34
215	56	10	v	-1
216	56	12	i	32
217	56	19	(	-1
218	56	20	)	-1
219	56	21	;	-1
220	57	2	O	-1
221	57	13	l	35
222	57	14	;	-1
223	58	2	O	-1
224	58	6	i	33
225	58	13	;	-1
226	59	2	O	-1
227	59	13	l	36
228	59	14	;	-1
229	60	2	O	-1
230	60	6	i	34
231	60	13	;	-1
232	61	1	r	-1
233	61	8	l	37
234	61	9	;	-1
235	62	1	}	-1

Листинг 2 – Таблица лексем

Позиция	Имя	Вид	Тип данных	Значение
0	CalculateF	Функция	Число	0
1	CalculateF_number	Параметр	Число	0
2	CalculateF_result	Переменная	Число	0
3	CalculateF_\$LEX1	Литерал	Число	1
4	main\$LEX4	Литерал	Строка	"-----"
5	main\$LEX5	Литерал	Строка	"Factorial:"
6	mainresult	Переменная	Число	0
7	main\$LEX6	Литерал	Число	3
8	main\$LEX7	Литерал	Число	4
9	main\$LEX8	Литерал	Строка	"!3 + !4"
10	mainbValue	Переменная	Логическая	0
11	main\$LEX9	Литерал	Логическая	1
12	main\$LEX11	Литерал	Строка	"bitwise operations in condition:"
13	main\$LEX12	Литерал	Число	66
14	main\$LEX13	Литерал	Число	11
15	main\$LEX14	Литерал	Число	72
16	main\$LEX15	Литерал	Число	80
17	main\$LEX16	Литерал	Число	2
18	main\$LEX18	Литерал	Строка	"((66 11) + 72) & (80 : (2 - 1)) = "
19	main\$LEX20	Литерал	Строка	"Char operations:"
20	mainc	Переменная	Символ	0
21	main\$LEX21	Литерал	Символ	Z
22	main\$LEX22	Литерал	Символ	z
23	main\$LEX23	Литерал	Строка	"'Z' - 'z':"
24	main\$LEX24	Литерал	Символ	a
25	main\$LEX25	Литерал	Символ	b
26	main\$LEX26	Литерал	Строка	"'a' + 'b':"
27	main\$LEX27	Литерал	Логическая	0
28	main\$LEX28	Литерал	Символ	A
29	main\$LEX29	Литерал	Символ	B
30	main\$LEX30	Литерал	Строка	"'A' + 'B':"
31	getDate	Функция	Строка	*пустая
строка*				
32	getTime	Функция	Строка	*пустая
строка*				
33	maindateNow	Переменная	Строка	*пустая
строка*				
34	maintimeNow	Переменная	Строка	*пустая
строка*				
35	main\$LEX32	Литерал	Строка	"date:"
36	main\$LEX33	Литерал	Строка	"time:"
37	main\$LEX34	Литерал	Число	0

Листинг 2 – Таблица идентификаторов

```

FST l_ubyte(
    str,
    6,
    NODE(1, RELATION('u', 1)),
    NODE(1, RELATION('b', 2)),
    NODE(1, RELATION('y', 3)),
    NODE(1, RELATION('t', 4)),
    NODE(1, RELATION('e', 5)),
    NODE()
);
FST l_until(
    str,
    6,
    NODE(1, RELATION('w', 1)),
    NODE(1, RELATION('h', 2)),
    NODE(1, RELATION('i', 3)),
    NODE(1, RELATION('l', 4)),
    NODE(1, RELATION('e', 5)),
    NODE()
);
FST l_do(
    str,
    3,
    NODE(1, RELATION('d', 1)),
    NODE(1, RELATION('o', 2)),
    NODE()
);
FST l_if(
    str,
    3,
    NODE(1, RELATION('i', 1)),
    NODE(1, RELATION('f', 2)),
    NODE()
);

FST l_add(
    str,
    4,
    NODE(1, RELATION('a', 1)),
    NODE(1, RELATION('d', 2)),
    NODE(1, RELATION('d', 3)),
    NODE()
);

FST l_str(
    str,
    7,
    NODE(1, RELATION('s', 1)),
    NODE(1, RELATION('t', 2)),
    NODE(1, RELATION('r', 3)),
    NODE(1, RELATION('i', 4)),
    NODE(1, RELATION('n', 5)),

```

```

        NODE(1, RELATION('g', 6)),
        NODE()
    );

FST l_bool(
    str,
    5,
    NODE(1, RELATION('b', 1)),
    NODE(1, RELATION('o', 2)),
    NODE(1, RELATION('o', 3)),
    NODE(1, RELATION('l', 4)),
    NODE()
);

FST l_true(
    str,
    5,
    NODE(1, RELATION('t', 1)),
    NODE(1, RELATION('r', 2)),
    NODE(1, RELATION('u', 3)),
    NODE(1, RELATION('e', 4)),
    NODE()
);

FST l_false(
    str,
    6,
    NODE(1, RELATION('f', 1)),
    NODE(1, RELATION('a', 2)),
    NODE(1, RELATION('l', 3)),
    NODE(1, RELATION('s', 4)),
    NODE(1, RELATION('e', 5)),
    NODE()
);

FST l_numberLiteral(
    str,
    2,
    NODE(20,

        RELATION('0', 0),
        RELATION('1', 0),
        RELATION('2', 0),
        RELATION('3', 0),
        RELATION('4', 0),
        RELATION('5', 0),
        RELATION('6', 0),
        RELATION('7', 0),
        RELATION('8', 0),
        RELATION('9', 0),

        RELATION('0', 1),
        RELATION('1', 1),
        RELATION('2', 1),

```

```

        RELATION('3', 1),
        RELATION('4', 1),
        RELATION('5', 1),
        RELATION('6', 1),
        RELATION('7', 1),
        RELATION('8', 1),
        RELATION('9', 1)
    ),
    NODE()
);
FST l_charLiteral(
    str,
    4,
    NODE(1, RELATION('\'', 1)),
    NODE(256,
        RELATION((char)39, 2),
        RELATION((char)0, 2),
        RELATION((char)1, 2),
        RELATION((char)2, 2),
        RELATION((char)3, 2),
        RELATION((char)4, 2),
        RELATION((char)5, 2),
        RELATION((char)6, 2),
        RELATION((char)7, 2),
        RELATION((char)8, 2),
        RELATION((char)9, 2),
        RELATION((char)10, 2),
        RELATION((char)11, 2),
        RELATION((char)12, 2),
        RELATION((char)13, 2),
        RELATION((char)14, 2),
        RELATION((char)15, 2),
        RELATION((char)16, 2),
        RELATION((char)17, 2),
        RELATION((char)18, 2),
        RELATION((char)19, 2),
        RELATION((char)20, 2),
        RELATION((char)21, 2),
        RELATION((char)22, 2),
        RELATION((char)23, 2),
        RELATION((char)24, 2),
        RELATION((char)25, 2),
        RELATION((char)26, 2),
        RELATION((char)27, 2),
        RELATION((char)28, 2),
        RELATION((char)29, 2),
        RELATION((char)30, 2),
        RELATION((char)31, 2),
        RELATION((char)32, 2),
        RELATION((char)33, 2),
        RELATION((char)34, 2),
        RELATION((char)35, 2),
        RELATION((char)36, 2),

```

```
RELATION((char)37, 2),  
RELATION((char)38, 2),  
RELATION((char)40, 2),  
RELATION((char)41, 2),  
RELATION((char)42, 2),  
RELATION((char)43, 2),  
RELATION((char)44, 2),  
RELATION((char)45, 2),  
RELATION((char)46, 2),  
RELATION((char)47, 2),  
RELATION((char)48, 2),  
RELATION((char)49, 2),  
RELATION((char)50, 2),  
RELATION((char)51, 2),  
RELATION((char)52, 2),  
RELATION((char)53, 2),  
RELATION((char)54, 2),  
RELATION((char)55, 2),  
RELATION((char)56, 2),  
RELATION((char)57, 2),  
RELATION((char)58, 2),  
RELATION((char)59, 2),  
RELATION((char)60, 2),  
RELATION((char)61, 2),  
RELATION((char)62, 2),  
RELATION((char)63, 2),  
RELATION((char)64, 2),  
RELATION((char)65, 2),  
RELATION((char)66, 2),  
RELATION((char)67, 2),  
RELATION((char)68, 2),  
RELATION((char)69, 2),  
RELATION((char)70, 2),  
RELATION((char)71, 2),  
RELATION((char)72, 2),  
RELATION((char)73, 2),  
RELATION((char)74, 2),  
RELATION((char)75, 2),  
RELATION((char)76, 2),  
RELATION((char)77, 2),  
RELATION((char)78, 2),  
RELATION((char)79, 2),  
RELATION((char)80, 2),  
RELATION((char)81, 2),  
RELATION((char)82, 2),  
RELATION((char)83, 2),  
RELATION((char)84, 2),  
RELATION((char)85, 2),  
RELATION((char)86, 2),  
RELATION((char)87, 2),  
RELATION((char)88, 2),  
RELATION((char)89, 2),  
RELATION((char)90, 2),
```

```
RELATION((char)91, 2),  
RELATION((char)92, 2),  
RELATION((char)93, 2),  
RELATION((char)94, 2),  
RELATION((char)95, 2),  
RELATION((char)96, 2),  
RELATION((char)97, 2),  
RELATION((char)98, 2),  
RELATION((char)99, 2),  
RELATION((char)100, 2),  
RELATION((char)101, 2),  
RELATION((char)102, 2),  
RELATION((char)103, 2),  
RELATION((char)104, 2),  
RELATION((char)105, 2),  
RELATION((char)106, 2),  
RELATION((char)107, 2),  
RELATION((char)108, 2),  
RELATION((char)109, 2),  
RELATION((char)110, 2),  
RELATION((char)111, 2),  
RELATION((char)112, 2),  
RELATION((char)113, 2),  
RELATION((char)114, 2),  
RELATION((char)115, 2),  
RELATION((char)116, 2),  
RELATION((char)117, 2),  
RELATION((char)118, 2),  
RELATION((char)119, 2),  
RELATION((char)120, 2),  
RELATION((char)121, 2),  
RELATION((char)122, 2),  
RELATION((char)123, 2),  
RELATION((char)124, 2),  
RELATION((char)125, 2),  
RELATION((char)126, 2),  
RELATION((char)127, 2),  
RELATION((char)128, 2),  
RELATION((char)129, 2),  
RELATION((char)130, 2),  
RELATION((char)131, 2),  
RELATION((char)132, 2),  
RELATION((char)133, 2),  
RELATION((char)134, 2),  
RELATION((char)135, 2),  
RELATION((char)136, 2),  
RELATION((char)137, 2),  
RELATION((char)138, 2),  
RELATION((char)139, 2),  
RELATION((char)140, 2),  
RELATION((char)141, 2),  
RELATION((char)142, 2),  
RELATION((char)143, 2),
```



```
RELATION((char)144, 2),  
RELATION((char)145, 2),  
RELATION((char)146, 2),  
RELATION((char)147, 2),  
RELATION((char)148, 2),  
RELATION((char)149, 2),  
RELATION((char)150, 2),  
RELATION((char)151, 2),  
RELATION((char)152, 2),  
RELATION((char)153, 2),  
RELATION((char)154, 2),  
RELATION((char)155, 2),  
RELATION((char)156, 2),  
RELATION((char)157, 2),  
RELATION((char)158, 2),  
RELATION((char)159, 2),  
RELATION((char)160, 2),  
RELATION((char)161, 2),  
RELATION((char)162, 2),  
RELATION((char)163, 2),  
RELATION((char)164, 2),  
RELATION((char)165, 2),  
RELATION((char)166, 2),  
RELATION((char)167, 2),  
RELATION((char)168, 2),  
RELATION((char)169, 2),  
RELATION((char)170, 2),  
RELATION((char)171, 2),  
RELATION((char)172, 2),  
RELATION((char)173, 2),  
RELATION((char)174, 2),  
RELATION((char)175, 2),  
RELATION((char)176, 2),  
RELATION((char)177, 2),  
RELATION((char)178, 2),  
RELATION((char)179, 2),  
RELATION((char)180, 2),  
RELATION((char)181, 2),  
RELATION((char)182, 2),  
RELATION((char)183, 2),  
RELATION((char)184, 2),  
RELATION((char)185, 2),  
RELATION((char)186, 2),  
RELATION((char)187, 2),  
RELATION((char)188, 2),  
RELATION((char)189, 2),  
RELATION((char)190, 2),  
RELATION((char)191, 2),  
RELATION((char)192, 2),  
RELATION((char)193, 2),  
RELATION((char)194, 2),  
RELATION((char)195, 2),  
RELATION((char)196, 2),
```

```
RELATION((char)197, 2),  
RELATION((char)198, 2),  
RELATION((char)199, 2),  
RELATION((char)200, 2),  
RELATION((char)201, 2),  
RELATION((char)202, 2),  
RELATION((char)203, 2),  
RELATION((char)204, 2),  
RELATION((char)205, 2),  
RELATION((char)206, 2),  
RELATION((char)207, 2),  
RELATION((char)208, 2),  
RELATION((char)209, 2),  
RELATION((char)210, 2),  
RELATION((char)211, 2),  
RELATION((char)212, 2),  
RELATION((char)213, 2),  
RELATION((char)214, 2),  
RELATION((char)215, 2),  
RELATION((char)216, 2),  
RELATION((char)217, 2),  
RELATION((char)218, 2),  
RELATION((char)219, 2),  
RELATION((char)220, 2),  
RELATION((char)221, 2),  
RELATION((char)222, 2),  
RELATION((char)223, 2),  
RELATION((char)224, 2),  
RELATION((char)225, 2),  
RELATION((char)226, 2),  
RELATION((char)227, 2),  
RELATION((char)228, 2),  
RELATION((char)229, 2),  
RELATION((char)230, 2),  
RELATION((char)231, 2),  
RELATION((char)232, 2),  
RELATION((char)233, 2),  
RELATION((char)234, 2),  
RELATION((char)235, 2),  
RELATION((char)236, 2),  
RELATION((char)237, 2),  
RELATION((char)238, 2),  
RELATION((char)239, 2),  
RELATION((char)240, 2),  
RELATION((char)241, 2),  
RELATION((char)242, 2),  
RELATION((char)243, 2),  
RELATION((char)244, 2),  
RELATION((char)245, 2),  
RELATION((char)246, 2),  
RELATION((char)247, 2),  
RELATION((char)248, 2),  
RELATION((char)249, 2),
```

```

        RELATION((char)250, 2),
        RELATION((char)251, 2),
        RELATION((char)252, 2),
        RELATION((char)253, 2),
        RELATION((char)254, 2),
        RELATION((char)255, 2)),
    NODE(1, RELATION('\'', 3)),
    NODE());

FST l_stringLiteral(
    str,
    3,
    NODE(1, RELATION('\\"', 1)),
    NODE(256,
        RELATION('\\"', 2),
        RELATION((char)0, 1),
        RELATION((char)1, 1),
        RELATION((char)2, 1),
        RELATION((char)3, 1),
        RELATION((char)4, 1),
        RELATION((char)5, 1),
        RELATION((char)6, 1),
        RELATION((char)7, 1),
        RELATION((char)8, 1),
        RELATION((char)9, 1),
        RELATION((char)10, 1),
        RELATION((char)11, 1),
        RELATION((char)12, 1),
        RELATION((char)13, 1),
        RELATION((char)14, 1),
        RELATION((char)15, 1),
        RELATION((char)16, 1),
        RELATION((char)17, 1),
        RELATION((char)18, 1),
        RELATION((char)19, 1),
        RELATION((char)20, 1),
        RELATION((char)21, 1),
        RELATION((char)22, 1),
        RELATION((char)23, 1),
        RELATION((char)24, 1),
        RELATION((char)25, 1),
        RELATION((char)26, 1),
        RELATION((char)27, 1),
        RELATION((char)28, 1),
        RELATION((char)29, 1),
        RELATION((char)30, 1),
        RELATION((char)31, 1),
        RELATION((char)32, 1),
        RELATION((char)33, 1),
        RELATION((char)35, 1),
        RELATION((char)36, 1),
        RELATION((char)37, 1),
        RELATION((char)38, 1),

```

```
RELATION((char)39, 1),  
RELATION((char)40, 1),  
RELATION((char)41, 1),  
RELATION((char)42, 1),  
RELATION((char)43, 1),  
RELATION((char)44, 1),  
RELATION((char)45, 1),  
RELATION((char)46, 1),  
RELATION((char)47, 1),  
RELATION((char)48, 1),  
RELATION((char)49, 1),  
RELATION((char)50, 1),  
RELATION((char)51, 1),  
RELATION((char)52, 1),  
RELATION((char)53, 1),  
RELATION((char)54, 1),  
RELATION((char)55, 1),  
RELATION((char)56, 1),  
RELATION((char)57, 1),  
RELATION((char)58, 1),  
RELATION((char)59, 1),  
RELATION((char)60, 1),  
RELATION((char)61, 1),  
RELATION((char)62, 1),  
RELATION((char)63, 1),  
RELATION((char)64, 1),  
RELATION((char)65, 1),  
RELATION((char)66, 1),  
RELATION((char)67, 1),  
RELATION((char)68, 1),  
RELATION((char)69, 1),  
RELATION((char)70, 1),  
RELATION((char)71, 1),  
RELATION((char)72, 1),  
RELATION((char)73, 1),  
RELATION((char)74, 1),  
RELATION((char)75, 1),  
RELATION((char)76, 1),  
RELATION((char)77, 1),  
RELATION((char)78, 1),  
RELATION((char)79, 1),  
RELATION((char)80, 1),  
RELATION((char)81, 1),  
RELATION((char)82, 1),  
RELATION((char)83, 1),  
RELATION((char)84, 1),  
RELATION((char)85, 1),  
RELATION((char)86, 1),  
RELATION((char)87, 1),  
RELATION((char)88, 1),  
RELATION((char)89, 1),  
RELATION((char)90, 1),  
RELATION((char)91, 1),
```

```
RELATION((char)92, 1),
RELATION((char)93, 1),
RELATION((char)94, 1),
RELATION((char)95, 1),
RELATION((char)96, 1),
RELATION((char)97, 1),
RELATION((char)98, 1),
RELATION((char)99, 1),
RELATION((char)100, 1),
RELATION((char)101, 1),
RELATION((char)102, 1),
RELATION((char)103, 1),
RELATION((char)104, 1),
RELATION((char)105, 1),
RELATION((char)106, 1),
RELATION((char)107, 1),
RELATION((char)108, 1),
RELATION((char)109, 1),
RELATION((char)110, 1),
RELATION((char)111, 1),
RELATION((char)112, 1),
RELATION((char)113, 1),
RELATION((char)114, 1),
RELATION((char)115, 1),
RELATION((char)116, 1),
RELATION((char)117, 1),
RELATION((char)118, 1),
RELATION((char)119, 1),
RELATION((char)120, 1),
RELATION((char)121, 1),
RELATION((char)122, 1),
RELATION((char)123, 1),
RELATION((char)124, 1),
RELATION((char)125, 1),
RELATION((char)126, 1),
RELATION((char)127, 1),
RELATION((char)128, 1),
RELATION((char)129, 1),
RELATION((char)130, 1),
RELATION((char)131, 1),
RELATION((char)132, 1),
RELATION((char)133, 1),
RELATION((char)134, 1),
RELATION((char)135, 1),
RELATION((char)136, 1),
RELATION((char)137, 1),
RELATION((char)138, 1),
RELATION((char)139, 1),
RELATION((char)140, 1),
RELATION((char)141, 1),
RELATION((char)142, 1),
RELATION((char)143, 1),
RELATION((char)144, 1),
```

```
RELATION((char)145, 1),  
RELATION((char)146, 1),  
RELATION((char)147, 1),  
RELATION((char)148, 1),  
RELATION((char)149, 1),  
RELATION((char)150, 1),  
RELATION((char)151, 1),  
RELATION((char)152, 1),  
RELATION((char)153, 1),  
RELATION((char)154, 1),  
RELATION((char)155, 1),  
RELATION((char)156, 1),  
RELATION((char)157, 1),  
RELATION((char)158, 1),  
RELATION((char)159, 1),  
RELATION((char)160, 1),  
RELATION((char)161, 1),  
RELATION((char)162, 1),  
RELATION((char)163, 1),  
RELATION((char)164, 1),  
RELATION((char)165, 1),  
RELATION((char)166, 1),  
RELATION((char)167, 1),  
RELATION((char)168, 1),  
RELATION((char)169, 1),  
RELATION((char)170, 1),  
RELATION((char)171, 1),  
RELATION((char)172, 1),  
RELATION((char)173, 1),  
RELATION((char)174, 1),  
RELATION((char)175, 1),  
RELATION((char)176, 1),  
RELATION((char)177, 1),  
RELATION((char)178, 1),  
RELATION((char)179, 1),  
RELATION((char)180, 1),  
RELATION((char)181, 1),  
RELATION((char)182, 1),  
RELATION((char)183, 1),  
RELATION((char)184, 1),  
RELATION((char)185, 1),  
RELATION((char)186, 1),  
RELATION((char)187, 1),  
RELATION((char)188, 1),  
RELATION((char)189, 1),  
RELATION((char)190, 1),  
RELATION((char)191, 1),  
RELATION((char)192, 1),  
RELATION((char)193, 1),  
RELATION((char)194, 1),  
RELATION((char)195, 1),  
RELATION((char)196, 1),  
RELATION((char)197, 1),
```

```
RELATION((char)198, 1),  
RELATION((char)199, 1),  
RELATION((char)200, 1),  
RELATION((char)201, 1),  
RELATION((char)202, 1),  
RELATION((char)203, 1),  
RELATION((char)204, 1),  
RELATION((char)205, 1),  
RELATION((char)206, 1),  
RELATION((char)207, 1),  
RELATION((char)208, 1),  
RELATION((char)209, 1),  
RELATION((char)210, 1),  
RELATION((char)211, 1),  
RELATION((char)212, 1),  
RELATION((char)213, 1),  
RELATION((char)214, 1),  
RELATION((char)215, 1),  
RELATION((char)216, 1),  
RELATION((char)217, 1),  
RELATION((char)218, 1),  
RELATION((char)219, 1),  
RELATION((char)220, 1),  
RELATION((char)221, 1),  
RELATION((char)222, 1),  
RELATION((char)223, 1),  
RELATION((char)224, 1),  
RELATION((char)225, 1),  
RELATION((char)226, 1),  
RELATION((char)227, 1),  
RELATION((char)228, 1),  
RELATION((char)229, 1),  
RELATION((char)230, 1),  
RELATION((char)231, 1),  
RELATION((char)232, 1),  
RELATION((char)233, 1),  
RELATION((char)234, 1),  
RELATION((char)235, 1),  
RELATION((char)236, 1),  
RELATION((char)237, 1),  
RELATION((char)238, 1),  
RELATION((char)239, 1),  
RELATION((char)240, 1),  
RELATION((char)241, 1),  
RELATION((char)242, 1),  
RELATION((char)243, 1),  
RELATION((char)244, 1),  
RELATION((char)245, 1),  
RELATION((char)246, 1),  
RELATION((char)247, 1),  
RELATION((char)248, 1),  
RELATION((char)249, 1),  
RELATION((char)250, 1),
```

```

        RELATION((char)251, 1),
        RELATION((char)252, 1),
        RELATION((char)253, 1),
        RELATION((char)254, 1),
        RELATION((char)255, 1)
    ),
    NODE()
);

```

```

FST l_itendificator(
    str,
    3,
    NODE(106,
        RELATION('a', 1),
        RELATION('b', 1),
        RELATION('c', 1),
        RELATION('d', 1),
        RELATION('e', 1),
        RELATION('f', 1),
        RELATION('g', 1),
        RELATION('h', 1),
        RELATION('i', 1),
        RELATION('j', 1),
        RELATION('k', 1),
        RELATION('l', 1),
        RELATION('m', 1),
        RELATION('n', 1),
        RELATION('o', 1),
        RELATION('p', 1),
        RELATION('q', 1),
        RELATION('r', 1),
        RELATION('s', 1),
        RELATION('t', 1),
        RELATION('u', 1),
        RELATION('v', 1),
        RELATION('w', 1),
        RELATION('x', 1),
        RELATION('y', 1),
        RELATION('z', 1),
        RELATION('A', 1),
        RELATION('B', 1),
        RELATION('C', 1),
        RELATION('D', 1),
        RELATION('E', 1),
        RELATION('F', 1),
        RELATION('G', 1),
        RELATION('H', 1),
        RELATION('I', 1),
        RELATION('J', 1),
        RELATION('K', 1),
        RELATION('L', 1),
        RELATION('M', 1),

```



```
RELATION('N', 1),
RELATION('O', 1),
RELATION('P', 1),
RELATION('Q', 1),
RELATION('R', 1),
RELATION('S', 1),
RELATION('T', 1),
RELATION('U', 1),
RELATION('V', 1),
RELATION('W', 1),
RELATION('X', 1),
RELATION('Y', 1),
RELATION('Z', 1),
RELATION('_', 1),
RELATION('a', 2),
RELATION('b', 2),
RELATION('c', 2),
RELATION('d', 2),
RELATION('e', 2),
RELATION('f', 2),
RELATION('g', 2),
RELATION('h', 2),
RELATION('i', 2),
RELATION('j', 2),
RELATION('k', 2),
RELATION('l', 2),
RELATION('m', 2),
RELATION('n', 2),
RELATION('o', 2),
RELATION('p', 2),
RELATION('q', 2),
RELATION('r', 2),
RELATION('s', 2),
RELATION('t', 2),
RELATION('u', 2),
RELATION('v', 2),
RELATION('w', 2),
RELATION('x', 2),
RELATION('y', 2),
RELATION('z', 2),
RELATION('A', 2),
RELATION('B', 2),
RELATION('C', 2),
RELATION('D', 2),
RELATION('E', 2),
RELATION('F', 2),
RELATION('G', 2),
RELATION('H', 2),
RELATION('I', 2),
RELATION('J', 2),
RELATION('K', 2),
RELATION('L', 2),
RELATION('M', 2),
```

```

        RELATION('N', 2),
        RELATION('O', 2),
        RELATION('P', 2),
        RELATION('Q', 2),
        RELATION('R', 2),
        RELATION('S', 2),
        RELATION('T', 2),
        RELATION('U', 2),
        RELATION('V', 2),
        RELATION('W', 2),
        RELATION('X', 2),
        RELATION('Y', 2),
        RELATION('Z', 2),
        RELATION('_', 2)
    ),
    NODE(126,
        RELATION('a', 1),
        RELATION('b', 1),
        RELATION('c', 1),
        RELATION('d', 1),
        RELATION('e', 1),
        RELATION('f', 1),
        RELATION('g', 1),
        RELATION('h', 1),
        RELATION('i', 1),
        RELATION('j', 1),
        RELATION('k', 1),
        RELATION('l', 1),
        RELATION('m', 1),
        RELATION('n', 1),
        RELATION('o', 1),
        RELATION('p', 1),
        RELATION('q', 1),
        RELATION('r', 1),
        RELATION('s', 1),
        RELATION('t', 1),
        RELATION('u', 1),
        RELATION('v', 1),
        RELATION('w', 1),
        RELATION('x', 1),
        RELATION('y', 1),
        RELATION('z', 1),
        RELATION('A', 1),
        RELATION('B', 1),
        RELATION('C', 1),
        RELATION('D', 1),
        RELATION('E', 1),
        RELATION('F', 1),
        RELATION('G', 1),
        RELATION('H', 1),
        RELATION('I', 1),
        RELATION('J', 1),
        RELATION('K', 1),

```

```
RELATION('L', 1),
RELATION('M', 1),
RELATION('N', 1),
RELATION('O', 1),
RELATION('P', 1),
RELATION('Q', 1),
RELATION('R', 1),
RELATION('S', 1),
RELATION('T', 1),
RELATION('U', 1),
RELATION('V', 1),
RELATION('W', 1),
RELATION('X', 1),
RELATION('Y', 1),
RELATION('Z', 1),
RELATION('_', 1),
RELATION('0', 1),
RELATION('1', 1),
RELATION('2', 1),
RELATION('3', 1),
RELATION('4', 1),
RELATION('5', 1),
RELATION('6', 1),
RELATION('7', 1),
RELATION('8', 1),
RELATION('9', 1),
RELATION('a', 2),
RELATION('b', 2),
RELATION('c', 2),
RELATION('d', 2),
RELATION('e', 2),
RELATION('f', 2),
RELATION('g', 2),
RELATION('h', 2),
RELATION('i', 2),
RELATION('j', 2),
RELATION('k', 2),
RELATION('l', 2),
RELATION('m', 2),
RELATION('n', 2),
RELATION('o', 2),
RELATION('p', 2),
RELATION('q', 2),
RELATION('r', 2),
RELATION('s', 2),
RELATION('t', 2),
RELATION('u', 2),
RELATION('v', 2),
RELATION('w', 2),
RELATION('x', 2),
RELATION('y', 2),
RELATION('z', 2),
RELATION('A', 2),
```

```

        RELATION('B', 2),
        RELATION('C', 2),
        RELATION('D', 2),
        RELATION('E', 2),
        RELATION('F', 2),
        RELATION('G', 2),
        RELATION('H', 2),
        RELATION('I', 2),
        RELATION('J', 2),
        RELATION('K', 2),
        RELATION('L', 2),
        RELATION('M', 2),
        RELATION('N', 2),
        RELATION('O', 2),
        RELATION('P', 2),
        RELATION('Q', 2),
        RELATION('R', 2),
        RELATION('S', 2),
        RELATION('T', 2),
        RELATION('U', 2),
        RELATION('V', 2),
        RELATION('W', 2),
        RELATION('X', 2),
        RELATION('Y', 2),
        RELATION('Z', 2),
        RELATION('_', 2),
        RELATION('0', 2),
        RELATION('1', 2),
        RELATION('2', 2),
        RELATION('3', 2),
        RELATION('4', 2),
        RELATION('5', 2),
        RELATION('6', 2),
        RELATION('7', 2),
        RELATION('8', 2),
        RELATION('9', 2)
    ),
    NODE()
);

FST l_string(
    str,
    5,
    NODE(1, RELATION('c', 1)),
    NODE(1, RELATION('h', 2)),
    NODE(1, RELATION('a', 3)),
    NODE(1, RELATION('r', 4)),
    NODE()
);

FST l_function(
    str,
    5,

```

```

        NODE(1, RELATION('f', 1)),
        NODE(1, RELATION('u', 2)),
        NODE(1, RELATION('n', 3)),
        NODE(1, RELATION('c', 4)),
        NODE()
    );
    FST l_declare(
        str,
        8,
        NODE(1, RELATION('d', 1)),
        NODE(1, RELATION('e', 2)),
        NODE(1, RELATION('c', 3)),
        NODE(1, RELATION('l', 4)),
        NODE(1, RELATION('a', 5)),
        NODE(1, RELATION('r', 6)),
        NODE(1, RELATION('e', 7)),
        NODE()
    );
    FST l_return(
        str,
        7,
        NODE(1, RELATION('r', 1)),
        NODE(1, RELATION('e', 2)),
        NODE(1, RELATION('t', 3)),
        NODE(1, RELATION('u', 4)),
        NODE(1, RELATION('r', 5)),
        NODE(1, RELATION('n', 6)),
        NODE()
    );
    FST l_write(
        str,
        6,
        NODE(1, RELATION('w', 1)),
        NODE(1, RELATION('r', 2)),
        NODE(1, RELATION('i', 3)),
        NODE(1, RELATION('t', 4)),
        NODE(1, RELATION('e', 5)),
        NODE()
    );
    FST l_out(
        str,
        4,
        NODE(1, RELATION('o', 1)),
        NODE(1, RELATION('u', 2)),
        NODE(1, RELATION('t', 3)),
        NODE()
    );
    FST l_main(
        str,
        5,
        NODE(1, RELATION('m', 1)),
        NODE(1, RELATION('a', 2)),

```

```

        NODE(1, RELATION('i', 3)),
        NODE(1, RELATION('n', 4)),
        NODE()
    );
    FST l_semicolon(
        str,
        2,
        NODE(1, RELATION(';', 1)),
        NODE()
    );
    FST l_comma(
        str,
        2,
        NODE(1, RELATION(',', 1)),
        NODE()
    );
    FST l_braceleft(
        str,
        2,
        NODE(1, RELATION('{', 1)),
        NODE()
    );
    FST l_braceright(
        str,
        2,
        NODE(1, RELATION('}', 1)),
        NODE()
    );
    FST l_lefthesis(
        str,
        2,
        NODE(1, RELATION('(', 1)),
        NODE()
    );
    FST l_cycleStart(
        str,
        2,
        NODE(1, RELATION('[', 1)),
        NODE()
    );
    FST l_cycleEnd(
        str,
        2,
        NODE(1, RELATION(']', 1)),
        NODE()
    );
    FST l_conditionalStart(
        str,
        2,
        NODE(1, RELATION('[', 1)),
        NODE()
    );
    FST l_conditionalEnd(

```

```

        str,
        2,
        NODE(1, RELATION(']', 1)),
        NODE()
    );
FST l_righththesis(
    str,
    2,
    NODE(1, RELATION(')', 1)),
    NODE()
);

FST l_verb(
    str,
    2,
    NODE(10, RELATION('+', 1), RELATION('-', 1), RELATION('*', 1),
1),
        RELATION('/', 1), RELATION(':', 1), RELATION('\\', 1),
RELATION('%', 1), RELATION('=', 1),
        RELATION('&', 1), RELATION('|', 1)),
    NODE()
);
FST l_inversion(
    str,
    2,
    NODE(1, RELATION('~', 1)),
    NODE()
);
FST l_boolVerb(
    str,
    2,
    NODE(4, RELATION('!', 1), RELATION('^', 1), RELATION('<', 1),
1), RELATION('>', 1)),
    NODE()
);

```

Листинг 3 – Конечные автоматы, соответствующие ключевым словам

```

#pragma once
#pragma once
#define ID_MAXSIZE      26
#define ID_CURRENT_MAXSIZE      12

#define TI_MAXSIZE      4096
#define TI_INT_DEFAULT  0x00000000
#define TI_STR_DEFAULT  0x00
#define TI_NULLIDX      0xffffffff
#define TI_STR_MAXSIZE  255

#define FALSYNUMBER 17

namespace IT
{
    enum IDDATATYPE { UBYTE = 1, STR = 2, CHR = 3, BOOL = 4 };
    enum IDTYPE { V = 1, F = 2, P = 3, L = 4 };

    struct Entry
    {
        int idxfirstLE;
        char id[ID_MAXSIZE];
        bool isExternal;
        IDDATATYPE iddatatype;
        IDTYPE idtype;
        union {
            unsigned int vubyte;
            char vchar;
            bool vbool;
            struct
            {
                char len;
                char* str;
            } vstr;
        } value;
        Entry(int idxfirstLE, const char* id, IDDATATYPE
iddatatype, IDTYPE idtype, bool e = false) {
            this->isExternal = e;
            this->idxfirstLE = idxfirstLE;
            this->iddatatype = iddatatype;
            this->idtype = idtype;
            if (iddatatype == CHR)
            {
                this->value.vchar = '\\0';
            }
            if (iddatatype == UBYTE)
            {
                this->value.vubyte = 0;
            }
            if (iddatatype == STR)
            {
                this->value.vstr.len = 0;
                this->value.vstr.str = nullptr;
            }
        }
    };
};

```



```

    }
    if (iddatatype == BOOL)
    {
        this->value.vbool = false;
    }

    int len = 0;
    for (int i = 0; id[i] != '\0' && i < ID_MAXSIZE;
i++)
    {
        len++;
        this->id[i] = id[i];
    }
    if (len >= ID_MAXSIZE)
        this->id[ID_MAXSIZE - 1] = '\0';
    else
    {
        this->id[len] = '\0';
    }

};

Entry(int idxfirstLE, const char* id, IDDATATYPE
iddatatype, IDTYPE idtype, unsigned int data, bool e = false) {
    this->isExternal = e;
    this->idxfirstLE = idxfirstLE;
    this->iddatatype = iddatatype;
    this->idtype = idtype;
    this->value.vubyte = data;
    int len = 0;
    for (int i = 0; id[i] != '\0' && i < ID_MAXSIZE;
i++)
    {
        len++;
        this->id[i] = id[i];
    }
    if (len >= ID_MAXSIZE)
        this->id[ID_MAXSIZE - 1] = '\0';
    else
    {
        this->id[len] = '\0';
    }

};

Entry(int idxfirstLE, const char* id, IDDATATYPE
iddatatype, IDTYPE idtype, char data, bool e = false) {
    this->isExternal = e;
    this->idxfirstLE = idxfirstLE;
    this->iddatatype = iddatatype;
    this->idtype = idtype;
    int len = 0;
    for (int i = 0; id[i] != '\0' && i < ID_MAXSIZE;
i++)
    {

```

```

        len++;
        this->id[i] = id[i];
    }
    if (len >= ID_MAXSIZE)
        this->id[ID_MAXSIZE - 1] = '\0';
    else
    {
        this->id[len] = '\0';
    }

    this->value.vchar = data;
};

Entry(int idxfirstLE, const char* id, IDDATATYPE
iddatatype, IDTYPE idtype, short data, bool e = false) {
    this->isExternal = e;
    this->idxfirstLE = idxfirstLE;
    this->iddatatype = iddatatype;
    this->idtype = idtype;
    int len = 0;
    for (int i = 0; id[i] != '\0' && i < ID_MAXSIZE;
i++)
    {
        len++;
        this->id[i] = id[i];
    }
    if (len >= ID_MAXSIZE)
        this->id[ID_MAXSIZE - 1] = '\0';
    else
    {
        this->id[len] = '\0';
    }

    this->value.vbool = data;
};

Entry(int idxfirstLE, const char* id, IDDATATYPE
iddatatype, IDTYPE idtype, char* data, bool e = false) {
    this->isExternal = e;
    this->idxfirstLE = idxfirstLE;
    this->iddatatype = iddatatype;
    this->idtype = idtype;
    int len = 0;
    for (int i = 0; id[i] != '\0' && i < ID_MAXSIZE;
i++)
    {
        len++;
        this->id[i] = id[i];
    }
    if (len >= ID_MAXSIZE)
        this->id[ID_MAXSIZE - 1] = '\0';
    else
    {
        this->id[len] = '\0';
    }
}

```

```

        len = 0;
        for (int i = 1; data[i] != '\"'; i++)
        {
            len++;
        }
        this->value.vstr.len = len + 3;
        this->value.vstr.str = new char[len + 3];

        for (int i = 0; i < len + 3; i++)
        {
            this->value.vstr.str[i] = data[i];
        }
        this->value.vstr.str[len + 3 - 1] = '\\0';
        this->value.vstr.len = len + 2;

    };
    Entry() = default;
};
struct IdTable
{
    int maxsize;
    int size;
    Entry* table;
};
IdTable Create(
    int size
);
void Add(
    IdTable& idtable,
    Entry entry
);
Entry GetEntry(
    IdTable& idtable,
    int n
);
int IsId(
    IdTable& idtable,
    char id[ID_MAXSIZE]
);
int IsLX(
    IdTable& idtable,
    IDDATATYPE type,
    unsigned int data
);
int IsLX(
    IdTable& idtable,
    IDDATATYPE type,
    short data
);
int IsLX(
    IdTable& idtable,
    IDDATATYPE type,

```

```
        char data
    );
    int IsLX(
        IdTable& idtable,
        IDDATATYPE type,
        char* data
    );

    bool isUniq(IdTable& idtable,
        char id[ID_MAXSIZE]);
    void Delete(IdTable& idtable);
}
```

Листинг 4 – Структура таблицы идентификаторов

```

#pragma once
#define LEXEMA_FIXSIZE 1
#define LT_MAXSIZE 4096
#define LT_TI_NULLIDX 0xFFFFFFFF
#define LEX_UBYTE 't'
#define LEX_STRING 't'
#define LEX_CHAR 't'
#define LEX_BOOL 't'
#define LEX_TYPE LEX_STRING
#define LEX_ID 'i'
#define LEX_UNTIL 'u'
#define LEX_DO 'D'
#define LEX_IF 'o'
#define LEX_MAIN 'm'
#define LEX_LITERAL 'l'
#define LEX_FUNCTION 'f'
#define LEX_DECLARE 'd'
#define LEX_RETURN 'r'
#define LEX_ADD 'a'
#define LEX_WRITE 'w'
#define LEX_OUT 'O'
#define LEX_SEMICOLON ';'
#define LEX_COMMA ','
#define LEX_LEFTBRACE '{'
#define LEX_RIGHTBRACE '}'

#define LEX_LEFT_SQUAREBRACE '['
#define LEX_RIGHT_SQUAREBRACE ']'

#define LEX_LEFTHESIS '('
#define LEX_RIGHTHESIS ')'
#define LEX_PLUS 'v'
#define LEX_MINUS 'v'
#define LEX_STAR 'v'
#define LEX_DIRSLASH 'v'
#define LEX_EQUAL 'v'
#define LEX_OPERATOR 'v'
#define LEX_OR 'v'
#define LEX_AND 'v'
#define LEX_INVERSION 'n'
#define LEX_BOOL_OPERATOR 'b'
#define EMPTY_DATA '\0'

namespace LT
{
    struct Entry
    {
        char lexema;
        int sn;
        int cn;
        int idxTI;
        char data;
    };
};

```

```

    Entry(
        char lexema,
        int sn,
        int cn,
        int idxTI,
        char symbol = EMPTY_DATA
    ) {
        this->lexema = lexema;
        this->sn = sn;
        this->cn = cn;
        this->idxTI = idxTI;
        this->data = data;
    };

    Entry() = default;
};

struct LexTable
{
    int maxsize;
    int size;
    Entry* table;
};

LexTable Create(int size);
void Add(LexTable& lextable, Entry entry);
Entry GetEntry(LexTable& lextable, int n);

void Delete(LexTable& lextable);
}

```

Листинг 5 – Структура таблицы лексем

ПРИЛОЖЕНИЕ Г

```

Greibach greibach(NS('S'), TS('$'),
    9,
    Rule(NS('S'), GRB_ERROR_SERIES + 0,
        7,
        Rule::Chain(7, TS('m'), TS('{'), NS('N'), TS('r'),
NS('E'), TS(';'), TS('}')),
        Rule::Chain(6, TS('m'), TS('{'), TS('r'), NS('E'),
TS(';'), TS('}')),
        Rule::Chain(15, TS('d'), TS('t'), TS('f'), TS('i'),
TS('('), NS('F'), TS(')'), TS('{'), NS('N'), TS('r'), NS('E'),
TS(';'), TS('}'), TS(';'), NS('S')),
        Rule::Chain(14, TS('d'), TS('t'), TS('f'), TS('i'),
TS('('), NS('F'), TS(')'), TS('{'), TS('r'), NS('E'), TS(';'),
TS('}'), TS(';'), NS('S')),
        Rule::Chain(14, TS('d'), TS('t'), TS('f'), TS('i'),
TS('('), TS(')'), TS('{'), NS('N'), TS('r'), NS('E'), TS(';'),
TS('}'), TS(';'), NS('S')),
        Rule::Chain(13, TS('d'), TS('t'), TS('f'), TS('i'),
TS('('), TS(')'), TS('{'), TS('r'), NS('E'), TS(';'), TS('}'),
TS(';'), NS('S')),
        Rule::Chain(6, TS('m'), TS('{'), TS('r'), NS('E'),
TS(';'), TS('}'))
    ),
    Rule(NS('N'), GRB_ERROR_SERIES + 1,
        19,
        Rule::Chain(4, TS('d'), TS('t'), TS('i'), TS(';')),
        Rule::Chain(6, TS('i'), TS('('), NS('F'), TS(')'),
TS(';'), NS('N')),
        Rule::Chain(5, TS('d'), TS('t'), TS('i'), TS(';'),
NS('N')),
        Rule::Chain(4, TS('i'), TS('v'), NS('E'), TS(';')),
        Rule::Chain(5, TS('i'), TS('v'), NS('E'), TS(';'),
NS('N')),
        Rule::Chain(9, TS('a'), TS('t'), TS('f'), TS('i'),
TS('('), NS('F'), TS(')'), TS(';'), NS('N')),
        Rule::Chain(8, TS('a'), TS('t'), TS('f'), TS('i'),
TS('('), NS('F'), TS(')'), TS(';')),
        Rule::Chain(8, TS('a'), TS('t'), TS('f'), TS('i'),
TS('('), TS(')'), TS(';'), NS('N')),
        Rule::Chain(7, TS('a'), TS('t'), TS('f'), TS('i'),
TS('('), TS(')'), TS(';')),
        Rule::Chain(4, TS('O'), TS('i'), TS(';'), NS('N')),
        Rule::Chain(3, TS('O'), TS('i'), TS(';')),
        Rule::Chain(4, TS('O'), TS('l'), TS(';'), NS('N')),
        Rule::Chain(3, TS('O'), TS('l'), TS(';')),
        Rule::Chain(8, TS('o'), TS('('), NS('B'), TS(')'),
TS('['), NS('N'), TS(']'), TS(';')),
        Rule::Chain(9, TS('o'), TS('('), NS('B'), TS(')'),
TS('['), NS('N'), TS(']'), TS(';'), NS('N')),
        Rule::Chain(8, TS('u'), TS('('), NS('B'), TS(')'),
TS('['), NS('N'), TS(']'), TS(';')),

```

```

        Rule::Chain(9, TS('u'), TS('('), NS('B'), TS(')'),
TS('[', NS('N'), TS(']'), TS(';'), NS('N')),
        Rule::Chain(10, TS('D'), TS('[', NS('N'), TS(']'),
TS('u'), TS('('), NS('B'), TS(')'), TS(';'), NS('N')),
        Rule::Chain(9, TS('D'), TS('[', NS('N'), TS(']'),
TS('u'), TS('('), NS('B'), TS(')'), TS(';'))
    ),
    Rule(NS('E'), GRB_ERROR_SERIES + 2,
        16,
        Rule::Chain(1, TS('i')),
        Rule::Chain(1, TS('l')),
        Rule::Chain(2, TS('n'), TS('i')),
        Rule::Chain(2, TS('n'), TS('l')),
        Rule::Chain(3, TS('('), NS('E'), TS(')'),
        Rule::Chain(4, TS('i'), TS('('), NS('W'), TS(')'),
        Rule::Chain(3, TS('i'), TS('('), TS(')'),
        Rule::Chain(2, TS('i'), NS('M')),
        Rule::Chain(2, TS('l'), NS('M')),
        Rule::Chain(3, TS('n'), TS('i'), NS('M')),
        Rule::Chain(3, TS('n'), TS('l'), NS('M')),
        Rule::Chain(4, TS('n'), TS('('), NS('E'), TS(')'),
        Rule::Chain(5, TS('n'), TS('('), NS('E'), TS(')'),
NS('M')),
        Rule::Chain(4, TS('('), NS('E'), TS(')'), NS('M')),
        Rule::Chain(4, TS('('), NS('E'), TS(')'), NS('M')),
        Rule::Chain(5, TS('i'), TS('('), NS('W'), TS(')'),
NS('M'))
    ),
    Rule(NS('M'), GRB_ERROR_SERIES + 3,
        4,
        Rule::Chain(2, TS('v'), NS('E')),
        Rule::Chain(4, TS('v'), TS('('), NS('E'), TS(')'),
        Rule::Chain(5, TS('v'), TS('('), NS('E'), TS(')'),
NS('M')),
        Rule::Chain(3, TS('v'), NS('E'), NS('M'))
    ),
    Rule(NS('F'), GRB_ERROR_SERIES + 4,
        5,
        Rule::Chain(2, TS('t'), TS('i')),
        Rule::Chain(4, TS('t'), TS('i'), TS(','), NS('F')),
        Rule::Chain(1, TS('i')),
        Rule::Chain(3, TS('i'), TS(','), NS('F')),
        Rule::Chain(3, TS('l'), TS(','), NS('F'))
    ),
    Rule(NS('W'), GRB_ERROR_SERIES + 5,
        4,
        Rule::Chain(1, TS('i')),
        Rule::Chain(1, TS('l')),
        Rule::Chain(3, TS('i'), TS(','), NS('W')),
        Rule::Chain(3, TS('l'), TS(','), NS('W'))
    ),
    Rule(NS('B'), GRB_ERROR_SERIES + 6,
        6,

```



```

        Rule::Chain(3, TS('i'), TS('b'), TS('i')),
        Rule::Chain(3, TS('i'), TS('b'), TS('l')),
        Rule::Chain(3, TS('l'), TS('b'), TS('i')),
        Rule::Chain(3, TS('l'), TS('b'), TS('l')),
        Rule::Chain(1, TS('i')),
        Rule::Chain(1, TS('l'))
    )
);

```

Листинг 1 – Грамматика языка SAA-2025

```

struct MfstDiagnosis
{
    short lenta_position; RC_STEP rc_step; short nrule;

    short nrule_chain;
    MfstDiagnosis();
    MfstDiagnosis(
        short plenta_position,
        RC_STEP prc_step,
        short pnrule,
        short pnrule_chain
    );
} diagnosis[MFST_DIAGN_NUMBER];

GRBALPHABET* lenta;
short lenta_position; short nrule; short nrulechain;
short lenta_size;
GRB::Greibach greibach;
LEX::LEX lex;
MFSTSTACK st;
bool shallWrite;
use_container<std::stack<MfstState>> storestate;
Mfst();
Mfst(
    LEX::LEX plex,
    GRB::Greibach pgreibach,
    bool shouldWrite
);
char* getCSt(char* buf);
char* getCLenta(char* buf, short pos, short n = 25);
char* getDiagnosis(short n, char* buf);
bool savestate(const Log::LOG& log);
bool reststate(const Log::LOG& log);
bool push_chain(
    GRB::Rule::Chain chain
);
RC_STEP step(const Log::LOG& log);
bool start(const Log::LOG& log);
bool savediagnosis(

```

```

        RC_STEP pprc_step
    );
void printrules(const Log::LOG& lo);
struct Deduction
{
    short size;
    short* nrules;
    short* nrulechains;
    Deduction() { size = 0; nrules = 0; nrulechains = 0; };
} deduction;
bool savededuction();

```

Листинг 2 – Структура конечного магазинного автомата

ПРИЛОЖЕНИЕ Д

```

1      : SAVESTATE:          1
1      :                    dtfi(ti){dti;ivl;D[ivivi;
dtfi(F){NrE;};S$
2      :                    tfi(ti){dti;ivl;D[ivivi;i
tfi(F){NrE;};S$
3      :                    fi(ti){dti;ivl;D[ivivi;iv
fi(F){NrE;};S$
4      :                    i(ti){dti;ivl;D[ivivi;ivi
i(F){NrE;};S$
5      :                    (ti){dti;ivl;D[ivivi;iviv
(F){NrE;};S$
6      :                    ti){dti;ivl;D[ivivi;ivivl
F){NrE;};S$
7      :F->ti                ti){dti;ivl;D[ivivi;ivivl
F){NrE;};S$
8      : SAVESTATE:          2
8      :                    ti){dti;ivl;D[ivivi;ivivl
ti){NrE;};S$
9      :                    i){dti;ivl;D[ivivi;ivivl;
i){NrE;};S$
10     :                    ){dti;ivl;D[ivivi;ivivl;O
){NrE;};S$
11     :                    {dti;ivl;D[ivivi;ivivl;Oi    {NrE;};S$
12     :                    dti;ivl;D[ivivi;ivivl;Oi;    NrE;};S$
13     :N->dti;                dti;ivl;D[ivivi;ivivl;Oi;    NrE;};S$
14     : SAVESTATE:          3
14     :                    dti;ivl;D[ivivi;ivivl;Oi;
dti;rE;};S$
15     :                    ti;ivl;D[ivivi;ivivl;Oi;]
ti;rE;};S$
16     :                    i;ivl;D[ivivi;ivivl;Oi;]u    i;rE;};S$
17     :                    ;ivl;D[ivivi;ivivl;Oi;]u(    ;rE;};S$
18     :                    ivl;D[ivivi;ivivl;Oi;]u(i    rE;};S$
19     : TS_NOK/NS_NORULECHAIN
19     : RESSTATE

```

Листинг 1 – Начало синтаксического анализа

```

1870:N->wi;          wl;];rl;}          N];rE;}$
1871: SAVESTATE:      75
1871:                wl;];rl;}          wi;];rE;}$
1872:                l;];rl;}          i;];rE;}$
1873: TS_NOK/NS_NORULECHAIN
1873: RESSTATE
1873:                wl;];rl;}          N];rE;}$
1874:N->wl;N         wl;];rl;}          N];rE;}$
1875: SAVESTATE:      75
1875:                wl;];rl;}          wl;N];rE;}$
1876:                l;];rl;}          l;N];rE;}$
1877:                ;];rl;}          ;N];rE;}$
1878:                ];rl;}          N];rE;}$
1879: TNS_NORULECHAIN/NS_NORULE
1879: RESSTATE
1879:                wl;];rl;}          N];rE;}$
1880:N->wl;          wl;];rl;}          N];rE;}$
1881: SAVESTATE:      75
1881:                wl;];rl;}          wl;];rE;}$
1882:                l;];rl;}          l;];rE;}$
1883:                ;];rl;}          ;];rE;}$
1884:                ];rl;}          ];rE;}$
1885:                ;rl;}          ;rE;}$
1886:                rl;}          rE;}$
1887:                l;}          E;}$
1888:E->l            l;}          E;}$
1889: SAVESTATE:      76
1889:                l;}          l;}$
1890:                ;}          ;}$
1891:                }          }$
1892:                $
1893: LENTA_END
1894: ----->LENTA_END

```

Листинг 2 – Конец синтаксического анализа

```

0   : S->dtfi (F) {NrE;};S
5   : F->ti
9   : N->dti;N
13  : N->ivE;N
15  : E->l
17  : N->u (B) [N];
19  : B->ibl
24  : N->ivE;N
26  : E->iM
27  : M->vE
28  : E->i
30  : N->ivE;
32  : E->iM
33  : M->vE
34  : E->l
39  : E->i
43  : S->m{NrE;}
45  : N->atfi ();N
52  : N->atfi ();N
59  : N->dti;N
63  : N->dti;N
67  : N->ivE;N
69  : E->i ()
73  : N->ivE;N
75  : E->i ()
79  : N->wi;N
82  : N->wi;N
85  : N->dti;N
89  : N->ivE;N
91  : E->lM
92  : M->vE
93  : E->l
95  : N->wl;N
98  : N->wi;N
101 : N->ivE;N
103 : E->lM
104 : M->vE
105 : E->l
107 : N->wl;N
110 : N->wi;N
113 : N->dti;N
117 : N->ivE;N
119 : E->i (W)M
121 : W->l
123 : M->vE
124 : E->i (W)
126 : W->l
129 : N->wl;N
132 : N->wi;N
135 : N->dti;N
139 : N->ivE;N
141 : E->l
143 : N->ivE;N

```

```
145 : E->n (E) M
147 : E->lM
148 : M->vE
149 : E->l
151 : M->vE
152 : E-> (E) M
153 : E->lM
154 : M->vE
155 : E->l
157 : M->vE
158 : E->l
160 : N->wl;N
163 : N->wi;N
166 : N->dti;N
170 : N->o (B) [N];N
172 : B->i
175 : N->wl;
180 : N->ivE;N
182 : E->l
184 : N->o (B) [N];
186 : B->i
189 : N->wl;
195 : E->l
```

Листинг 3 – Пример разбора синтаксическим анализатором

ПРИЛОЖЕНИЕ Е

```
#include <stack>
#include <vector>
#include <iostream>
#include "PolishNotation.h"
#include "Error.h"

namespace PolishNotation {
    template <typename T>
    struct container : T
    {
        using T::T;
        using T::c;
    };

    std::string toString(int n) {
        char buf[40];
        sprintf_s(buf, "%d", n);
        return buf;
    }

    bool find_elem(std::stack<char> stack, size_t size, char
elem) {
        for (size_t i = 0; i < size; i++)
            if (stack.top() == elem)
                return true;
            else
                stack.pop();
        return false;
    }

    int get_priority(char a)
    {
        switch (a)
        {
            case '(':
                return 0;
            case ')':
                return 0;
            case ',':
                return 1;
            case '-':
                return 2;
            case '+':
                return 2;
            case '*':
                return 3;
            case '%':
                return 3;
            case '/':
                return 3;
            case '\\':
                return 3;
        }
    }
}
```

```

        case ':':
            return 3;
        case '&':
            return 3;
        case '|':
            return 3;
        case '~':
            return 4;
        default: {
            return 0;
        }
    }

    void fixIt(LT::LexTable& lextable, const std::string& str,
size_t length, size_t pos, const std::vector<int>& ids) {
        std::cout << str << std::endl;
        for (size_t i = 0, q = 0; i < str.size(); i++) {
            lextable.table[pos + i].lexema = str[i];
            if (lextable.table[pos + i].lexema == LEX_ID ||
lextable.table[pos + i].lexema == LEX_LITERAL) {
                lextable.table[pos + i].idxTI = ids[q];
                q++;
            }
            else
                lextable.table[pos + i].idxTI =
LT_TI_NULLIDX;
        }
        int temp = str.size() + pos;
        for (size_t i = 0; i < length - str.size(); i++) {
            lextable.table[temp + i].idxTI = LT_TI_NULLIDX;
            lextable.table[temp + i].lexema = '#';
            lextable.table[temp + i].sn = -1;
        }
    }

    bool PolishNotation(int lextable_pos, LT::LexTable&
lextable, IT::IdTable& idtable)
    {
        container<std::stack<char>> stack;
        std::string PolishString;
        std::vector<int> ids;
        int operators_count = 0, operands_count = 0, iterator =
0, right_counter = 0, left_counter = 0, params_counter = 0;

        for (int i = lextable_pos; i < lextable.size; i++,
iterator++) {
            char lexem = lextable.table[i].lexema;
            char data = lextable.table[i].data;
            size_t stack_size = stack.size();
            if (idtable.table[lextable.table[i].idxTI].idtype
== IT::IDTYPE::F) {
                stack.push('@');
            }
        }
    }

```

```

        operands_count--;
    }
    switch (lexem) {
    case LEX_OPERATOR:
    {
        if (!stack.empty() && stack.top() !=
LEX_LEFTHESIS) {
            while (!stack.empty() &&
get_priority(data) <= get_priority(stack.top())) {
                PolishString += stack.top();
                stack.pop();
            }
            stack.push(data);
            operators_count++;
            break;
        }
    case LEX_INVERSION:
    {
        stack.push(data);
        break;
    }
    case LEX_COMMA:
    {
        while (!stack.empty()) {
            if (stack.top() == LEX_LEFTHESIS)
                break;
            PolishString += stack.top();
            stack.pop();
        }
        operands_count--;
        break;
    }
    case LEX_LEFTHESIS:
    {
        left_counter++;
        stack.push(lexem);
        break;
    }
    case LEX_RIGHTHESIS:
    {
        right_counter++;
        if (!find_elem(stack, stack_size,
LEX_LEFTHESIS))
            return false;
        while (stack.top() != LEX_LEFTHESIS) {
            PolishString += stack.top();
            stack.pop();
        }
        stack.pop();
        if (!stack.empty() && stack.top() == '@') {
            PolishString += stack.top() +
toString(params_counter - 1);

```



```

        params_counter = 0;
        stack.pop();
    }
    break;
}
case LEX_SEMICOLON:
{
    if (operators_count != 0 && operands_count !=
0)
        if ((!stack.empty() && (stack.top() ==
LEX_RIGHTTHESIS || stack.top() == LEX_LEFTTHESIS))
            || right_counter != left_counter ||
operands_count - operators_count != 1)
            return false;
        while (!stack.empty()) {
            PolishString += stack.top();
            stack.pop();
        }
        fixIt(lextable, PolishString, iterator,
lextable_pos, ids);
        return true;
        break;
    }
    case LEX_ID: {
        if (std::find(stack.c.begin(),
stack.c.begin(), '@') != stack.c.end())
            params_counter++;
        PolishString += lexem;
        if (lextable.table[i].idxTI != LT_TI_NULLIDX)
            ids.push_back(lextable.table[i].idxTI);
        operands_count++;
        if (!stack.empty() && stack.top() == '~')
        {
            PolishString += stack.top();
            stack.pop();
        }
        break;
    }
    case LEX_LITERAL: {
        if (std::find(stack.c.begin(),
stack.c.begin(), '@') != stack.c.end())
            params_counter++;
        PolishString += lexem;
        if (lextable.table[i].idxTI != LT_TI_NULLIDX)
            ids.push_back(lextable.table[i].idxTI);
        operands_count++;
        if (!stack.empty() && stack.top() == '~')
        {
            PolishString += stack.top();
            stack.pop();
        }
        break;
    }
}

```

```

        }

    }
    return true;
}

void DoPolish(LEX::LEX t) {
    for (int i = 0; i < t.lextable.size; i++)
        if (t.lextable.table[i].lexema == LEX_EQUAL)
            if (!PolishNotation(i + 1, t.lextable,
t.idtable))
                throw ERROR_THROW(130);
    for (int i = 0; i < t.lextable.size; i++)
        if (t.lextable.table[i].lexema == '+' ||
t.lextable.table[i].lexema == '-' || t.lextable.table[i].lexema
== '*' ||
            t.lextable.table[i].lexema == '/' ||
t.lextable.table[i].lexema == '\\' || t.lextable.table[i].lexema
== ':' ||
            t.lextable.table[i].lexema == '%' ||
t.lextable.table[i].lexema == '&' || t.lextable.table[i].lexema
== '|'
            || t.lextable.table[i].lexema == '~')
        {
            t.lextable.table[i].data =
t.lextable.table[i].lexema;
            t.lextable.table[i].lexema = LEX_OPERATOR;
        }
    }
}

```

Листинг 1 – алгоритм преобразования выражений к обратной польской записи, реализованный на C++

ПРИЛОЖЕНИЕ Ж

```

.586
.model flat, stdcall
includelib libcrt.lib
includelib kernel32.lib
includelib ../Debug/SAA-2025LIB.lib
ExitProcess PROTO :DWORD

    CurrentDate PROTO
    CurrentTime PROTO
outputubyte PROTO :BYTE
outputchar PROTO :BYTE
outputstr PROTO :DWORD

outputbool PROTO :BYTE

.stack 4096
.const
divideOnZeroExeption BYTE "Trying devide on zero", 0
    CalculateFac_$LEX1 BYTE 1 ;UBYTE
    main$LEX4 BYTE "-----", 0 ;STR
    main$LEX5 BYTE "Factorial:", 0 ;STR
    main$LEX6 BYTE 3 ;UBYTE
    main$LEX7 BYTE 4 ;UBYTE
    main$LEX8 BYTE "!3 + !4", 0 ;STR
    main$LEX9 BYTE 1 ;BOOL
    main$LEX11 BYTE "bitwise operations in condition:", 0 ;STR
    main$LEX12 BYTE 66 ;UBYTE
    main$LEX13 BYTE 11 ;UBYTE
    main$LEX14 BYTE 72 ;UBYTE
    main$LEX15 BYTE 80 ;UBYTE
    main$LEX18 BYTE "((66 | 11) + 72) & (80 : (2 - 1)) = ", 0
;STR
    main$LEX20 BYTE "Char operations:", 0 ;STR
    main$LEX21 BYTE 'Z' ;CHR
    main$LEX22 BYTE 'z' ;CHR
    main$LEX23 BYTE "'Z' - 'z':", 0 ;STR
    main$LEX24 BYTE 'a' ;CHR
    main$LEX25 BYTE 'b' ;CHR
    main$LEX26 BYTE "'a' + 'b':", 0 ;STR
    main$LEX27 BYTE 0 ;BOOL
    main$LEX28 BYTE 'A' ;CHR
    main$LEX29 BYTE 'B' ;CHR
    main$LEX30 BYTE "'A' + 'B':", 0 ;STR
    main$LEX32 BYTE "date:", 0 ;STR
    main$LEX33 BYTE "time:", 0 ;STR
    main$LEX34 BYTE 0 ;UBYTE
.data
    CalculateFac_result BYTE 0 ;UBYTE
    mainresult BYTE 0 ;UBYTE
    mainbValue BYTE 0 ;BOOL

```

```

        mainc BYTE 0 ;CHR
        maindateNow DWORD 0 ;STR
        maintimeNow DWORD 0 ;STR

.code
$CalculateFac PROC uses ebx ecx edi esi ,    CalculateFac_number:
BYTE
; String #4 :ivl
movzx eax, CalculateFac_$LEX1
push eax
pop eax
mov CalculateFac_result, al

DoWhile17Start:

; String #7 :iviiv
movzx eax, CalculateFac_result
push eax
movzx eax, CalculateFac_number
push eax
pop ebx
pop eax
mul ebx
push eax
pop eax
mov CalculateFac_result, al

; String #8 :ivilv
movzx eax, CalculateFac_number
push eax
movzx eax, CalculateFac_$LEX1
push eax
pop ebx
pop eax
sub eax, ebx
push eax
pop eax
mov CalculateFac_number, al
push eax
movzx eax, CalculateFac_number
push eax
CALL outputubyte
pop eax

movzx eax, CalculateFac_number
movzx ebx, CalculateFac_$LEX1
cmp eax, ebx
ja DoWhile17Start
DoWhile17End:

movzx eax, CalculateFac_result
ret
$CalculateFac ENDP

```

```

main PROC

push offset main$LEX4
CALL outputstr

push offset main$LEX5
CALL outputstr

; String #18 :ivil@1il@1v
invoke $CalculateFac, main$LEX6
push eax ;результат функции
invoke $CalculateFac, main$LEX7
push eax ;результат функции
pop ebx
pop eax
add eax, ebx
push eax
pop eax
mov mainresult, al

push offset main$LEX8
CALL outputstr
push eax
movzx eax, mainresult
push eax
CALL outputbyte
pop eax

; String #23 :ivl
movzx eax, main$LEX9
push eax
pop eax
mov mainbValue, al

If85Start:
movzx eax, mainbValue
mov ebx, 1
cmp eax, ebx
jnz If85End

push offset main$LEX4
CALL outputstr

push offset main$LEX11
CALL outputstr

; String #28 :ivllvllvlllvvv#####
movzx eax, main$LEX12
push eax
movzx eax, main$LEX13
push eax

```

```

pop ebx
pop eax
or eax, ebx
push eax
movzx eax, main$LEX14
push eax
pop ebx
pop eax
add eax, ebx
push eax
movzx eax, main$LEX15
push eax
movzx eax, CalculateFac_$LEX1
push eax
movzx eax, CalculateFac_$LEX1
push eax
pop ebx
pop eax
sub eax, ebx
push eax
pop ebx
pop eax
push edx ; сохраняем данные регистра edx
mov edx, 0
TEST  EBX, EBX
JZ     div_by_0
div ebx
pop edx
push eax
pop ebx
pop eax
and eax, ebx
push eax
pop eax
mov mainresult, al

push offset main$LEX18
CALL outputstr
push eax
movzx eax, mainresult
push eax
CALL outputbyte
pop eax

If85End:

push offset main$LEX4
CALL outputstr

push offset main$LEX20
CALL outputstr

; String #36 :ivllv

```

```

movzx eax, main$LEX21
push eax
movzx eax, main$LEX22
push eax
pop ebx
pop eax
sub eax, ebx
push eax
pop eax
mov mainc, al

push offset main$LEX23
CALL outputstr
push eax
movzx eax, mainc
push eax
CALL outputchar
pop eax

; String #39 :ivllv
movzx eax, main$LEX24
push eax
movzx eax, main$LEX25
push eax
pop ebx
pop eax
add eax, ebx
push eax
pop eax
mov mainc, al

push offset main$LEX26
CALL outputstr
push eax
movzx eax, mainc
push eax
CALL outputchar
pop eax

; String #42 :ivl
movzx eax, main$LEX27
push eax
pop eax
mov mainbValue, al

If164Start:
movzx eax, mainbValue
mov ebx, 1
cmp eax, ebx
jnz If164End

```

```

; String #45 :ivllv
movzx eax, main$LEX28
push eax
movzx eax, main$LEX29
push eax
pop ebx
pop eax
add eax, ebx
push eax
pop eax
mov mainc, al

push offset main$LEX30
CALL outputstr
push eax
movzx eax, mainc
push eax
CALL outputchar
pop eax

If164End:

push offset main$LEX4
CALL outputstr

; String #55 :ivi@0
invoke CurrentDate
push eax ;результат функции
pop maindateNow

; String #56 :ivi@0
invoke CurrentTime
push eax ;результат функции
pop maintimeNow

push offset main$LEX32
CALL outputstr

push maindateNow
CALL outputstr

push offset main$LEX33
CALL outputstr

push maintimeNow
CALL outputstr

movzx eax, main$LEX34
    jmp endPoint
div_by_0:
    push offset divideOnZeroExeption
CALL outputstr
endPoint:

```



```
        invoke      ExitProcess, eax  
main ENDP  
end main
```

Листинг 1 – код, сгенерированный транслятором